

DATABASE MANAGEMENT SYSTEM

UNIT 1 — INTRODUCTION TO DATABASE SYSTEMS

A. VERY SHORT QUESTIONS — 2 MARKS

Q1. Define Database. [2 Marks]

A **database** is an organized collection of related data that is stored and managed in a structured manner. It allows data to be easily accessed, retrieved, updated, and managed.

Q2. What is DBMS? [2 Marks]

DBMS (Database Management System) is software that provides an interface for creating, storing, retrieving, updating, and managing data in a database.

Examples: MySQL, Oracle, PostgreSQL.

Q3. What is a Database System? [2 Marks]

A **database system** consists of the database itself together with the DBMS, applications, users, and other components required to store, manage, and access data.

Q4. What is Data Management? [2 Marks]

Data management is the process of collecting, storing, organizing, maintaining, securing, and retrieving data so that it can be used effectively.

Q5. What is Logical Structure of a Database? [2 Marks]

The **logical structure** describes how data is organized from the user's or application's point of view, including tables, attributes, relationships, and constraints.

Q6. What is Physical Structure of a Database? [2 Marks]

The **physical structure** describes how database data is actually stored on physical storage devices, such as files, blocks, and storage locations.

Q7. Define Relational Database. [2 Marks]

A **relational database** stores data in the form of tables consisting of rows and columns. Relationships between tables are established using keys.

Q8. Define Distributed Database. [2 Marks]

A **distributed database** is a database whose data is stored across multiple physically separated locations connected through a network, while being managed as a single database system.

B. SHORT QUESTIONS — 7 MARKS

Q9. Explain the Importance of Data and Data Management. [7 Marks]

Data is an important resource for an organization because it provides the information required for daily operations, planning, and decision-making. **Data management** ensures that data remains organized, accurate, secure, and available when required.

Importance of Data and Data Management:

1. **Supports decision-making:**
Properly managed data provides useful information for making effective business and organizational decisions.
2. **Improves data accessibility:**
Data can be searched and retrieved efficiently whenever it is required.

3. **Maintains data accuracy:**
Proper management helps maintain correct and reliable data.
4. **Reduces data redundancy:**
Effective database management minimizes unnecessary duplication of the same data.
5. **Provides data security:**
Access to important data can be controlled according to user permissions.
6. **Maintains data consistency:**
Data remains consistent across different parts of the database.
7. **Supports organizational operations:**
Organizations can efficiently manage customers, employees, transactions, records, and other important information.

Conclusion:

Effective data management makes organizational data **organized, accurate, accessible, consistent, and secure**, allowing it to be used efficiently.

Q10. Explain the Functions of DBMS. [7 Marks]

A DBMS provides various functions for managing data efficiently.

Major functions of DBMS:

1. **Data Definition:**
Allows users to define databases, tables, attributes, relationships, and constraints.
2. **Data Storage:**
Stores data in an organized manner so that it can be efficiently accessed.
3. **Data Retrieval:**
Allows users to search and retrieve required information from the database.
4. **Data Manipulation:**
Provides operations for inserting, updating, and deleting data.
5. **Security and Authorization:**
Controls who can access particular data and what operations they are allowed to perform.
6. **Integrity Management:**
Helps maintain the correctness and consistency of data through constraints and rules.
7. **Backup and Recovery:**
Provides mechanisms to recover data after system failures or other problems.

Conclusion:

A DBMS provides a controlled and efficient environment for **storing, accessing, modifying, securing, and maintaining data**.

Q11. Explain the Advantages of DBMS. [7 Marks]

The major advantages of using a DBMS are:

1. **Reduced data redundancy:**
It reduces unnecessary duplication of data.
2. **Improved data consistency:**
Changes to data can be managed systematically, helping maintain consistency.
3. **Data security:**
Authorization and access control protect data from unauthorized users.
4. **Data sharing:**
Multiple authorized users and applications can access the same database.
5. **Data integrity:**
Constraints and rules help maintain accurate and valid data.
6. **Backup and recovery:**
Data can be backed up and recovered when failures occur.
7. **Easy data access:**
Users can efficiently search and retrieve required information.

Conclusion:

DBMS provides a more **secure, consistent, organized, and efficient** method of managing data than traditional file-based approaches.

Q12. Differentiate Between Logical and Physical Structure of a Database. [7 Marks]

Basis	Logical Structure	Physical Structure
Meaning	Describes the logical organization of data.	Describes the actual storage of data.
Concern	Tables, attributes, relationships, constraints.	Files, blocks, storage locations, etc.
View	Mainly concerned with user/application view.	Mainly concerned with storage.
Storage details	Hides physical storage details.	Deals with actual storage details.

Basis	Logical Structure	Physical Structure
Purpose	Defines how data is organized logically.	Defines how data is stored physically.
Change	Can be modified without necessarily changing physical storage.	Changes involve physical storage organization.
Example	Student table with ID, name, and address.	The actual files/blocks where student records are stored.

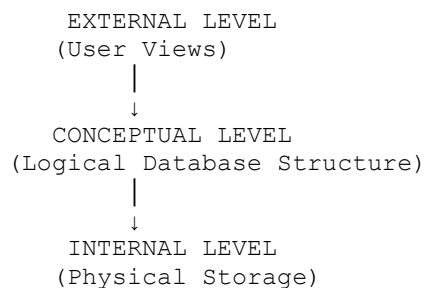
In simple words:

Logical structure = How data is organized and viewed.

Physical structure = How data is actually stored.

Q13. Explain the Three-Level Database Architecture. [7 Marks] ★ ★ ★

The **three-level database architecture** divides the database into three levels to separate the user view, logical organization, and physical storage of data.



1. External Level

- It is the highest level of database architecture.
- It represents the view of individual users or applications.
- Different users can have different views of the same database.
- It hides unnecessary database details from users.

2. Conceptual Level

- It represents the overall logical structure of the database.

- It describes entities, attributes, relationships, and constraints.
- It provides a complete logical view of the database.
- It does not deal with the details of physical storage.

3. Internal Level

- It is the lowest level.
- It describes how data is physically stored.
- It deals with storage structures and physical organization of data.

Importance:

This architecture separates **user views, logical organization, and physical storage**, making database management easier and reducing dependence between different levels.

Q14. Explain How Data is Stored in a Database. [7 Marks]

A database stores related data in an organized structure so that it can be accessed and managed efficiently.

In a relational database, data is generally organized as:

```

Database
  ↓
Tables
  ↓
Rows + Columns
  ↓
Individual Data

```

Main components:

1. **Database:**
A collection of related data managed as a whole.
2. **Table:**
Stores related data in a structured form.
3. **Row/Record:**
Represents one individual record in a table.
4. **Column/Field:**
Represents a particular attribute or property of the data.
5. **Keys:**
Keys identify records and can be used to establish relationships between tables.
6. **Relationships:**
Tables can be related to one another through appropriate keys.
7. **Storage:**
The DBMS manages the physical storage of the database on storage devices.

Thus, the DBMS hides complex storage details and provides users with an organized way to access the stored data.

Q15. Differentiate Between Relational Database and Distributed Database. [7 Marks] ★ ★ ★

Relational Database

Stores data primarily in related tables.

Uses rows and columns to organize data.

Relationships are established between tables.

Can be implemented at a single location.

SQL is commonly used for data manipulation.

Main focus is relational organization of data.

Example: a database containing related Student and Course tables.

Distributed Database

Stores data across multiple physical locations.

Data may be distributed among different network sites.

Different sites cooperate to provide access to distributed data.

Designed to operate across multiple locations.

Requires mechanisms for managing data across sites.

Main focus is distribution of data across locations.

Example: organizational data stored in databases at different branches.

Key difference:

Relational database → focuses on how data is organized into related tables.

Distributed database → focuses on storing and managing data across multiple locations.

C. LONG QUESTIONS — 15 MARKS

Q16. Explain DBMS, Its Functions and Advantages. [15 Marks] ★ ★ ★

Introduction

A **Database Management System (DBMS)** is software used to create, store, organize, retrieve, update, and manage data in a database. It provides an interface between users/applications and the database.

A DBMS allows organizations to manage large amounts of data efficiently while maintaining security, consistency, and integrity.

Functions of DBMS

1. Data Definition

DBMS allows users to define the structure of a database, including tables, attributes, relationships, and constraints.

2. Data Storage

It manages the storage of data in an organized manner and provides mechanisms for accessing stored information.

3. Data Retrieval

Users can retrieve required information from the database using appropriate queries.

4. Data Manipulation

A DBMS supports operations such as inserting, updating, and deleting records.

5. Security and Authorization

It controls access to the database and prevents unauthorized users from accessing restricted information.

6. Integrity Management

It applies rules and constraints to maintain the accuracy and consistency of data.

7. Backup and Recovery

It provides mechanisms for backing up data and recovering it after failures.

Advantages of DBMS

1. **Reduces data redundancy** by minimizing unnecessary duplication.
2. **Improves data consistency** by managing data systematically.
3. **Provides data security** through authorization and access control.
4. **Allows data sharing** among authorized users.
5. **Maintains data integrity** using constraints and rules.
6. **Provides backup and recovery** facilities.
7. **Makes data retrieval easier and faster.**
8. **Provides centralized management** of database resources.
9. **Supports multiple users and applications.**
10. **Provides data independence** by separating data organization from applications.

Conclusion

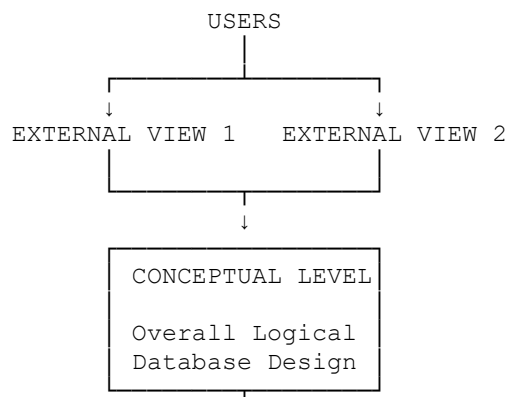
A DBMS provides an efficient and controlled environment for managing organizational data. It improves **security, consistency, integrity, accessibility, and reliability** of data and is therefore an essential component of modern database systems.

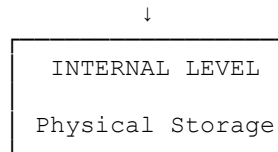
Q17. Explain Three-Level Database Architecture with a Suitable Diagram. [15 Marks] ★ ★ ★

Introduction

Database architecture describes how a database is organized and viewed at different levels. The three-level architecture separates the database into **external, conceptual, and internal levels**.

Diagram





1. External Level

The external level is the **user view** of the database.

- It is the highest level.
- Different users can have different views.
- Each user sees only the information relevant to their work.
- Unnecessary database details are hidden.

For example, a student may see their marks and personal information, while an administrator may see additional student records.

2. Conceptual Level

The conceptual level represents the **complete logical structure** of the database.

It describes:

- Entities
- Attributes
- Relationships
- Constraints
- Overall database organization

It does not describe how data is physically stored.

3. Internal Level

The internal level represents the **physical storage** of the database.

It describes:

- How data is stored
- Storage structures
- Files and blocks
- Physical organization of records

Advantages of Three-Level Architecture

1. **Data abstraction:** Users do not need to know physical storage details.
2. **Data independence:** Changes at one level can be made with less effect on other levels.
3. **Multiple user views:** Different users can have different views.

4. **Improved security:** Users can be restricted to appropriate views.
5. **Simplifies database management:** Each level has a specific responsibility.

Conclusion

The three-level architecture provides separation between the **external user view**, **conceptual logical structure**, and **internal physical storage**. It makes databases easier to manage and provides data abstraction and independence.

Q18. Explain the Evolution/History of Database Systems. [15 Marks]

Database systems have evolved as the amount and complexity of data increased.

1. Traditional File-Based Systems

Initially, organizations stored data in separate files.

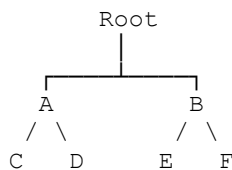
Problems:

- Data redundancy
- Data inconsistency
- Difficult data sharing
- Limited security
- Difficult data management

These problems created the need for specialized database systems.

2. Hierarchical Database Systems

The hierarchical model organized data in a **tree-like structure**.



It was suitable for data with parent-child relationships but was less flexible for complex relationships.

3. Network Database Systems

The network model allowed records to have more complex relationships than the hierarchical model.

It could represent **many-to-many relationships** more effectively.

4. Relational Database Systems

The relational model organized data into **tables consisting of rows and columns**.

It became widely used because of:

- Simple tabular structure
- Relationships between tables
- Powerful query capabilities
- Easier data management

5. Distributed and Modern Database Systems

As organizations expanded and required access to data from multiple locations, distributed database systems developed.

Data can be stored at different physical locations while being managed as part of a database system.

Summary of Evolution

```
File-Based
  ↓
Hierarchical
  ↓
Network
  ↓
Relational
  ↓
Distributed / Modern Database Systems
```

Conclusion

The evolution of database systems occurred mainly because organizations required better **data storage, organization, sharing, retrieval, security, and management**. Modern systems provide more powerful and flexible methods of managing large amounts of distributed data.

UNIT 2 — RELATIONAL DATA MODELS

A. VERY SHORT QUESTIONS — 2 MARKS

Q1. What is an Entity? [2 Marks]

An **entity** is a real-world object or concept that can be uniquely identified and about which data is stored in a database.

Example: Student, Employee, Customer.

Q2. What is an Attribute? [2 Marks]

An **attribute** is a property or characteristic that describes an entity.

Example: For Student, attributes may be Student_ID, Name, Address, and Age.

Q3. What is a Relationship? [2 Marks]

A **relationship** represents an association between two or more entities in a database.

Example: A Student **enrolls in** a Course.

Q4. What is an ER Model? [2 Marks]

The **Entity-Relationship (ER) model** is a conceptual data model used to represent entities, their attributes, and relationships among entities.

Q5. What is an ER Diagram? [2 Marks]

An **ER diagram** is a graphical representation of an ER model that shows entities, attributes, and relationships using standard symbols.

Q6. What is a Relational Model? [2 Marks]

The **relational model** represents data using tables called **relations**. Each table consists of rows and columns, and relationships between tables are established using keys.

B. SHORT QUESTIONS — 7 MARKS

Q7. Explain the Components of an ER Model. [7 Marks] ★ ★ ★

The major components of an **Entity-Relationship model** are entities, attributes, and relationships.

1. Entity

An entity represents a real-world object or concept about which information is stored.

Example: Student, Teacher, Course.

2. Attribute

An attribute describes a property of an entity.

For example, the attributes of Student may include:

Student_ID, Name, Address, Age

3. Relationship

A relationship represents an association between entities.

For example:

Student — enrolls in — Course

4. Entity Set

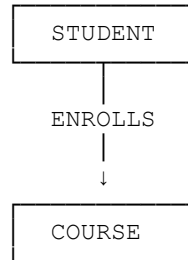
An entity set is a collection of similar entities.

For example, all students in a college form a **Student entity set**.

5. Relationship Set

A relationship set is a collection of similar relationships between entity sets.

ER Representation



Thus, the ER model provides a simple way of representing the **objects, properties, and associations** of a database.

Q8. Explain the Importance of an ER Diagram. [7 Marks] ★ ★ ★

An ER diagram is an important tool for designing and understanding a database.

Importance:

1. **Provides visual representation:**
It represents entities, attributes, and relationships graphically.
2. **Simplifies database design:**
Complex database requirements can be represented in an easier form.
3. **Identifies entities and relationships:**
It helps designers identify the important objects and their associations.
4. **Improves communication:**
Developers, analysts, and users can understand the database structure more easily.
5. **Helps identify missing information:**
During design, missing entities, attributes, or relationships can be detected.
6. **Acts as a database blueprint:**
It provides a foundation for converting the conceptual design into tables.
7. **Reduces design errors:**
Problems in relationships and data organization can be identified before implementation.

Conclusion:

An ER diagram helps create a **clear, organized, and accurate database design** before the database is actually implemented.

Q9. Explain the Relational Model. [7 Marks] ★ ★ ★

The **relational model** is a database model in which data is represented using tables called **relations**.

A relation consists of **rows and columns**.

Main components:

1. **Relation/Table:**
A table used to store related data.
2. **Tuple/Row:**
A row represents one complete record.
3. **Attribute/Column:**
A column represents a property of the data.
4. **Domain:**
The set of valid values that an attribute can contain.
5. **Primary Key:**
An attribute or combination of attributes that uniquely identifies each row.
6. **Foreign Key:**
An attribute that references a key in another table and is used to establish relationships.

Example:

STUDENT			
Student_ID	Name	Age	
101	Ram	20	
102	Sita	21	

Here:

- **STUDENT** = Relation/Table
- **Student_ID, Name, Age** = Attributes
- Each horizontal record = Tuple/Row
- **Student_ID** can act as the Primary Key.

Conclusion:

The relational model provides a **simple tabular structure** for storing and managing data and is widely used in database systems.

Q10. Explain the Advantages and Disadvantages of the ER Data Model. [7 Marks] ★ ★ ★

Advantages

1. **Simple representation:**
ER diagrams provide an easy graphical representation of database structure.
2. **Easy to understand:**
Users and developers can understand entities and relationships easily.
3. **Useful for database design:**
It provides a foundation for designing the database.
4. **Improves communication:**
It provides a common representation for users, analysts, and developers.
5. **Helps identify design problems:**
Missing or incorrect relationships can be detected during the design stage.

Disadvantages

1. **Can become complex:**
Large databases may require very complicated ER diagrams.
2. **Time-consuming:**
Designing a detailed ER model may require significant time.
3. **Limited implementation details:**
An ER diagram mainly represents conceptual structure and does not describe all physical implementation details.

Conclusion:

The ER model is highly useful for **conceptual database design**, but large and complex systems can make ER diagrams difficult to manage.

C. LONG QUESTIONS — 15 MARKS

Q11. Explain the ER Model and ER Diagram with Their Components and Importance. [15 Marks]



Introduction

The **Entity-Relationship (ER) model** is a conceptual data model used to represent the structure of a database. It identifies the important entities, their attributes, and the relationships among them.

An **ER diagram** provides a graphical representation of this model.

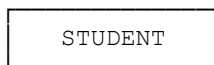
Components of ER Model

1. Entity

An entity is a real-world object or concept that can be uniquely identified.

Examples: Student, Employee, Product.

In an ER diagram, an entity is generally represented by a **rectangle**.



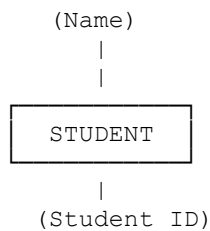
2. Attribute

An attribute is a property that describes an entity.

For example, Student may have:

- Student_ID
- Name
- Address
- Age

Attributes are generally represented by **ovals**.



3. Relationship

A relationship represents an association between two or more entities.

For example:



A relationship is generally represented using a **diamond** in a traditional ER diagram.

4. Entity Set

An entity set is a collection of similar entities.

For example, all students registered in a college form a **Student entity set**.

5. Relationship Set

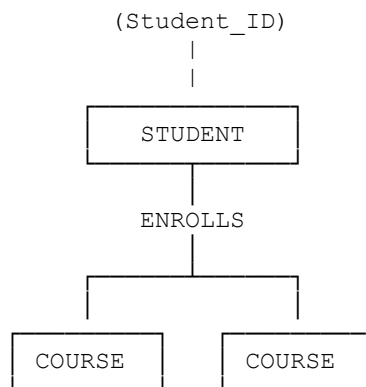
A relationship set is a collection of similar relationships among entity sets.

Importance of ER Diagram

1. **Provides graphical representation** of the database.
 2. **Simplifies database design.**
 3. Helps identify **entities and relationships**.
 4. Improves communication between **users and developers**.
 5. Helps identify **missing or incorrect information**.
 6. Acts as a **blueprint for database implementation**.
 7. Helps reduce **design errors**.
 8. Makes complex database requirements easier to understand.
-

Example

Consider a college database:



A student can enroll in courses, establishing a relationship between the **Student** and **Course** entities.

Conclusion

The ER model is an important conceptual modeling technique. It represents **entities, attributes, and relationships** clearly and provides a foundation for designing an effective database.

Q12. Explain the Relational Model with Its Components, Advantages and Example. [15 Marks]



Introduction

The **relational model** is a database model that organizes data into tables called relations. Each relation contains rows and columns, and relationships between tables can be established using keys.

Components of Relational Model

1. Relation

A relation is a table containing related data.

Example:

STUDENT			
Student_ID	Name	Age	
101	Ram	20	
102	Sita	21	
103	Hari	20	

2. Tuple

A tuple is a **row** in a relation. It represents one complete record.

3. Attribute

An attribute is a **column** in a relation and represents a property of an entity.

4. Domain

A domain defines the set of valid values that an attribute can contain.

5. Primary Key

A primary key uniquely identifies each tuple in a relation.

Example:

Student_ID can be the primary key.

6. Foreign Key

A foreign key is an attribute that references a key in another table and establishes a relationship between tables.

Advantages of Relational Model

1. **Simple structure:** Data is organized into understandable tables.
 2. **Easy data access:** Data can be queried efficiently.
 3. **Reduces redundancy:** Proper table design can minimize duplicate data.
 4. **Maintains integrity:** Constraints help maintain valid and accurate data.
 5. **Supports relationships:** Tables can be connected using keys.
 6. **Flexible querying:** Users can retrieve required data using query languages.
 7. **Easy maintenance:** Changes to data can be managed systematically.
-

Example of Related Tables

```
STUDENT
+-----+-----+
| Student_ID | Name |
+-----+-----+
| 101        | Ram  |
| 102        | Sita |
+-----+-----+
      |
      | Student_ID
      ↓
ENROLLMENT
+-----+-----+
| Student_ID | Course_ID |
+-----+-----+
| 101        | C01       |
| 102        | C02       |
+-----+-----+
```

The **Student_ID** can connect the two tables.

Conclusion

The relational model provides a simple and structured method of organizing data into tables. The use of **keys, relationships, and constraints** makes it effective for database management.

UNIT 3 — RELATIONAL ALGEBRA & CALCULUS

A. VERY SHORT QUESTIONS — 2 MARKS

Q1. What is Relational Algebra? [2 Marks]

Relational Algebra is a procedural query language used to retrieve and manipulate data from relational databases. It uses operators to perform operations on relations and produces a relation as the result.

Q2. What is a Unary Operator? [2 Marks]

A **unary operator** operates on a single relation.

Examples:

- Selection (σ)
 - Projection (π)
 - Rename (ρ)
-

Q3. What is a Binary Operator? [2 Marks]

A **binary operator** operates on two relations.

Examples:

- Union ($\cup$)
 - Intersection ($\cap$)
 - Set Difference ($-$)
 - Cartesian Product ($\times$)
 - Join
-

Q4. What is Selection? [2 Marks]

Selection (σ) retrieves rows from a relation that satisfy a specified condition.

Notation:

σ condition (Relation)

Example:

σ Age > 20 (STUDENT)

Q5. What is Projection? [2 Marks]

Projection (π) retrieves specific columns/attributes from a relation.

Notation:

π attributes (Relation)

Example:

π Name, Age (STUDENT)

Q6. What is Cartesian Product? [2 Marks]

Cartesian Product ($\times$) combines every tuple of one relation with every tuple of another relation.

Notation:

$R \times S$

Q7. What is Natural Join? [2 Marks]

A **Natural Join** combines two relations based on their common attributes and automatically removes duplicate common columns from the result.

Symbol:

$R \bowtie S$

Q8. What is Set Difference? [2 Marks]

Set Difference ($-$) returns the tuples that exist in the first relation but do not exist in the second relation.

$R - S$

B. SHORT QUESTIONS — 7 MARKS

Q9. Explain the Fundamental Operations of Relational Algebra. [7 Marks] ★ ★ ★

Relational algebra provides operations for retrieving and manipulating data in relational databases.

1. Selection (σ)

Selects rows that satisfy a given condition.

$\sigma \text{ condition } (R)$

Example:

$\sigma \text{ Age } > 20 \text{ (STUDENT)}$

2. Projection (π)

Selects required columns from a relation.

$\pi \text{ Name, Age (STUDENT)}$

3. Union ($\cup$)

Combines tuples from two compatible relations and eliminates duplicate tuples.

$R \cup S$

4. Set Difference ($-$)

Returns tuples that exist in R but not in S.

$R - S$

5. Cartesian Product ($\times$)

Combines every tuple of R with every tuple of S.

$R \times S$

6. Rename (ρ)

Changes the name of a relation or its attributes.

ρ NewName (R)

7. Join ($\bowtie$)

Combines related tuples from two relations based on a specified condition.

$R \bowtie S$

Conclusion

These operations form the basic foundation of **relational algebra** and are used to retrieve and manipulate information from relational databases.

Q10. Explain Selection and Projection with Examples. [7 Marks] ★ ★ ★

Selection and projection are two important **unary operations** in relational algebra.

Consider:

STUDENT

ID	Name	Age	City
1	Ram	20	Itahari
2	Sita	22	Dharan
3	Hari	19	Biratn

1. Selection (σ)

Selection retrieves **rows** that satisfy a condition.

Example:

σ Age > 20 (STUDENT)

Result:

ID	Name	Age	City
2	Sita	22	Dharan

2	Sita	22	Dharan
---	------	----	--------

Thus, selection filters **rows**.

2. Projection (π)

Projection retrieves specified **columns**.

Example:

π Name, City (STUDENT)

Result:

Name	City
Ram	Itahari
Sita	Dharan
Hari	Biratn

Thus, projection selects **columns**.

Main Difference

Selection → selects rows.

Projection → selects columns.

Q11. Explain Union, Intersection and Set Difference. [7 Marks] ★ ★ ★

These are set operations performed on two relations.

1. Union ($\cup$)

Union returns all tuples that appear in either relation, removing duplicates.

$R \cup S$

For union, the two relations should be **union-compatible**.

2. Intersection ($\cap$)

Intersection returns tuples that are common to both relations.

$$R \cap S$$

Example:

$$R = \{1, 2, 3\}$$

$$S = \{2, 3, 4\}$$

$$R \cap S = \{2, 3\}$$

3. Set Difference ($-$)

Set difference returns tuples present in the first relation but not in the second.

$$R - S$$

Example:

$$R = \{1, 2, 3\}$$

$$S = \{2, 3, 4\}$$

$$R - S = \{1\}$$

Similarly:

$$S - R = \{4\}$$

Summary

Operation Result

$R \cup S$ Tuples in R or S

$R \cap S$ Tuples common to R and S

$R - S$ Tuples in R but not S

Q12. Explain Cartesian Product and Join Operations. [7 Marks] ★ ★ ★

1. Cartesian Product

Cartesian product combines every tuple of one relation with every tuple of another relation.

$$R \times S$$

If R contains **m** tuples and S contains **n** tuples, the result contains:

$m \times n$ tuples

Example:

$$R = \{A, B\}$$

$$S = \{1, 2\}$$

$$R \times S =$$

$$\{(A, 1), (A, 2), (B, 1), (B, 2)\}$$

2. Join

Join combines related tuples from two relations according to a condition.

$$R \bowtie S$$

Common types included in your syllabus are:

- **Theta Join**
- **Equijoin**
- **Natural Join**

Theta Join

A theta join uses a comparison condition such as:

$$=, <, >, \leq, \geq, \neq$$

$$R \bowtie \text{condition } S$$

Equijoin

Equijoin is a theta join that uses only the **equal (=)** condition.

Natural Join

Natural join joins relations using common attributes and removes duplicate common columns.

Conclusion

Cartesian product produces combinations of tuples, while joins combine **related tuples based on conditions**.

Q13. Differentiate Between Unary and Binary Relational Algebra Operators. [7 Marks]

Unary Operators

Operate on one relation.

Require only one input relation.

Used for filtering, selecting, or renaming data.

Selection is unary.

Projection is unary.

Rename is unary.

Example: σ , π , ρ

Binary Operators

Operate on two relations.

Require two input relations.

Used for combining or comparing relations.

Union is binary.

Intersection is binary.

Cartesian product is binary.

Example: $\cup$, $\cap$, $-$, $\times$, $\bowtie$

Conclusion:

Unary $\rightarrow$ one relation

Binary $\rightarrow$ two relations

C. LONG QUESTIONS — 15 MARKS

Q14. Explain the Fundamental Relational Algebra Operations with Suitable Examples. [15 Marks]



Introduction

Relational Algebra is a procedural query language used to retrieve and manipulate data in relational databases. Its operators take one or more relations as input and produce a relation as output.

The fundamental operations are described below.

1. Selection (σ)

Selection retrieves only the tuples that satisfy a specified condition.

σ condition (R)

Example:

σ Age > 20 (STUDENT)

It returns students whose age is greater than 20.

Selection operates on rows.

2. Projection (π)

Projection retrieves specified attributes from a relation.

π Name, Age (STUDENT)

It returns only the Name and Age columns.

Projection operates on columns.

3. Rename (ρ)

Rename changes the name of a relation or its attributes.

ρ Student_Info (STUDENT)

This changes the relation name from STUDENT to Student_Info.

4. Union ($\cup$)

Union combines tuples from two union-compatible relations and removes duplicates.

R $\cup$ S

The relations must have compatible attributes.

5. Intersection ($\cap$)

Intersection returns tuples that are present in both relations.

$$R \cap S$$

Example:

$$R = \{1, 2, 3\}$$

$$S = \{2, 3, 4\}$$

$$R \cap S = \{2, 3\}$$

6. Set Difference ($-$)

Set difference returns tuples that belong to the first relation but not to the second.

$$R - S$$

Example:

$$R = \{1, 2, 3\}$$

$$S = \{2, 3, 4\}$$

$$R - S = \{1\}$$

7. Cartesian Product ($\times$)

Cartesian product combines every tuple of one relation with every tuple of another.

$$R \times S$$

If R has m tuples and S has n tuples, the result has:

m $\times$ n tuples

8. Theta Join

Theta join combines tuples from two relations based on a comparison condition.

$$R \bowtie \text{condition } S$$

The condition can use:

$=, <, >, \leq, \geq, \neq$

9. Equijoin

Equijoin is a special type of theta join where the condition uses only equality.

$R \bowtie_{R.A = S.A} S$

It combines records where the specified attributes are equal.

10. Natural Join

Natural join combines two relations based on common attributes.

$R \bowtie S$

It automatically removes duplicate copies of the common attribute.

11. Division

Division is used when a query requires finding entities that are related to **all** entities in another relation.

It is represented as:

$R \div S$

For example, it can be used to find students who have completed **all required courses**.

Summary Table

Operation	Symbol	Main Purpose
Selection	σ	Select rows
Projection	π	Select columns

Operation	Symbol	Main Purpose
Rename	ρ	Rename relation/attributes
Union	$\cup$	Combine relations
Intersection	$\cap$	Find common tuples
Difference	$-$	Find tuples in first only
Cartesian Product	$\times$	Combine every tuple
Theta Join	$\bowtie_{\theta}$	Join using condition
Equijoin	$\bowtie$	Join using equality
Natural Join	$\bowtie$	Join using common attributes
Division	$\div$	Find "all" relationships

Conclusion

Relational algebra provides a formal set of operations for querying relational databases. **Selection, projection, set operations, Cartesian product, joins, and division** form the foundation for retrieving and manipulating relational data.

Q15. Explain Selection, Projection, Join and Division with Suitable Examples. [15 Marks] ★ ★ ★

These operations are particularly important for retrieving meaningful information from multiple relations.

1. Selection

Selection filters tuples according to a condition.

$\sigma_{\text{Age} > 20}(\text{STUDENT})$

It returns only students whose age is greater than 20.

2. Projection

Projection selects required attributes.

π Name, Address (STUDENT)

It returns only the Name and Address attributes.

3. Join

Join combines related tuples from two relations.

Consider:

STUDENT		
ID	Name	Course_ID
1	Ram	C01
2	Sita	C02

COURSE	
Course_ID	Course_Name
C01	DBMS
C02	AI

A natural join can combine them:

STUDENT $\bowtie$ COURSE

Result:

ID	Name	Course_ID	Course_Name
1	Ram	C01	DBMS
2	Sita	C02	AI

4. Division

Division is used to answer queries involving "all".

For example:

Find students who have completed all courses.

If:

ENROLLMENT (Student, Course)

and

REQUIRED_COURSE (Course)

then:

ENROLLMENT $\div$ REQUIRED_COURSE

returns students who are associated with **every course** in REQUIRED_COURSE.

Conclusion

Selection, projection, join, and division allow relational algebra to answer different types of queries, from simple filtering to complex "**all condition**" queries.

UNIT 4 — SQL

A. VERY SHORT QUESTIONS — 2 MARKS

Q1. What is SQL? [2 Marks]

SQL (Structured Query Language) is a standard language used to create, manipulate, retrieve, and control data in relational databases.

Q2. What is DDL? Give examples. [2 Marks]

DDL (Data Definition Language) is used to define and modify the structure of database objects such as tables.

Examples: CREATE, ALTER, DROP, TRUNCATE.

Q3. What is DML? Give examples. [2 Marks]

DML (Data Manipulation Language) is used to insert, retrieve, modify, and delete data stored in database tables.

Examples: SELECT, INSERT, UPDATE, DELETE.

Q4. What is DCL? Give examples. [2 Marks]

DCL (Data Control Language) is used to control access and permissions on database objects.

Examples: GRANT, REVOKE.

Q5. What is the use of WHERE clause? [2 Marks]

The **WHERE** clause is used to specify a condition for selecting rows from a table.

```
SELECT * FROM Student
WHERE age > 20;
```

Q6. What is the use of ORDER BY? [2 Marks]

The **ORDER BY** clause is used to sort query results in ascending or descending order.

```
SELECT * FROM Student
ORDER BY name ASC;
```

Q7. What is BETWEEN in SQL? [2 Marks]

BETWEEN is used to select values that fall within a specified range, including the boundary values.

```
SELECT * FROM Student
WHERE age BETWEEN 18 AND 25;
```

Q8. What is LIKE in SQL? [2 Marks]

LIKE is used for pattern matching in character/string values.

```
SELECT * FROM Student
WHERE name LIKE 'A%';
```

Here, % represents zero or more characters.

Q9. What is a Subquery? [2 Marks]

A **subquery** is a query written inside another SQL query. The inner query provides a result that is used by the outer query.

B. SHORT QUESTIONS — 7 MARKS

Q10. Explain the Types of SQL Commands. [7 Marks]



SQL commands can be broadly categorized according to their purpose.

1. DDL — Data Definition Language

DDL is used to define and modify the structure of database objects.

Commands:

- CREATE
- ALTER
- DROP
- TRUNCATE

Example:

```
CREATE TABLE Student (  
    id INT,  
    name VARCHAR(50)  
);
```

2. DML — Data Manipulation Language

DML is used to manipulate data stored in database tables.

Commands:

- SELECT
- INSERT
- UPDATE
- DELETE

Example:

```
INSERT INTO Student  
VALUES (1, 'Ram');
```

3. DCL — Data Control Language

DCL controls access and permissions on database objects.

Commands:

- GRANT
- REVOKE

Example:

```
GRANT SELECT ON Student TO user1;
```

Summary

Type	Full Form	Main Purpose	Examples
DDL	Data Definition Language	Define database structure	CREATE, ALTER, DROP
DML	Data Manipulation Language	Manipulate data	SELECT, INSERT, UPDATE, DELETE
DCL	Data Control Language	Control access	GRANT, REVOKE

Q11. Explain Row Selection and Search Conditions in SQL. [7 Marks] ★ ★ ★

Row selection means retrieving only those rows that satisfy a specified condition. SQL uses the `WHERE` clause to perform row selection.

Syntax

```
SELECT column1, column2
FROM table_name
WHERE condition;
```

Common comparison operators

- = Equal to
- < Less than
- > Greater than
- <= Less than or equal to
- >= Greater than or equal to
- <> or != Not equal to

Examples

```
SELECT * FROM Student
WHERE age = 20;
SELECT * FROM Student
WHERE age > 20;
SELECT name FROM Student
WHERE age <= 25;
```

Logical operators

Conditions can be combined using:

AND

```
WHERE age > 18 AND city = 'Itahari';
```

OR

```
WHERE city = 'Itahari' OR city = 'Dharan';
```

NOT

```
WHERE NOT city = 'Kathmandu';
```

Conclusion

The WHERE clause and search conditions allow users to retrieve **only the required rows** from a table.

Q12. Explain BETWEEN, IN, LIKE and IS NULL with Examples. [7 Marks] ★ ★ ★

SQL provides special operators for convenient searching and filtering.

1. BETWEEN

Used to select values within a specified range.

```
SELECT * FROM Student  
WHERE age BETWEEN 18 AND 25;
```

It includes the boundary values.

2. IN

Used to compare a value against a list of specified values.

```
SELECT * FROM Student  
WHERE city IN ('Itahari', 'Dharan', 'Biratnagar');
```

This returns students from any of the specified cities.

3. LIKE

Used for pattern matching.

```
SELECT * FROM Student
WHERE name LIKE 'A%';
```

Common wildcards:

- % → zero or more characters
- _ → exactly one character

Example:

```
WHERE name LIKE '_am';
```

Can match a three-character name ending in am.

4. IS NULL

Used to check whether a column contains a NULL value.

```
SELECT * FROM Student
WHERE phone IS NULL;
```

NULL represents a missing or unknown value.

Summary

Operator Purpose

BETWEEN Select values within a range

IN Match values from a list

LIKE Pattern matching

IS NULL Check for NULL values

Q13. Explain ORDER BY and Duplicate Rows in SQL. [7 Marks]

ORDER BY

The ORDER BY clause is used to sort the result of a query.

Syntax

```
SELECT columns  
FROM table  
ORDER BY column [ASC|DESC];
```

Ascending Order

```
SELECT * FROM Student  
ORDER BY name ASC;
```

ASC sorts from smallest to largest or A to Z. It is the default ordering.

Descending Order

```
SELECT * FROM Student  
ORDER BY age DESC;
```

DESC sorts from largest to smallest or Z to A.

Sorting by Multiple Columns

```
SELECT * FROM Student  
ORDER BY city ASC, age DESC;
```

Duplicate Rows

SQL may return duplicate values when the selected column contains repeated values.

For example:

```
SELECT city FROM Student;
```

may return:

```
Itahari  
Dharan  
Itahari  
Biratnagar
```

To remove duplicates, use `DISTINCT`:

```
SELECT DISTINCT city  
FROM Student;
```

Result:

```
Itahari  
Dharan  
Biratnagar
```

Conclusion

`ORDER BY` controls the **sorting of query results**, while `DISTINCT` can be used to **remove duplicate rows from the result**.

Q14. Explain Multi-Table Queries in SQL. [7 Marks]



A **multi-table query** retrieves data from two or more related tables. Tables are usually connected using a common attribute or key.

Consider:

STUDENT		
ID	Name	Course_ID
1	Ram	C01
2	Sita	C02

COURSE	
Course_ID	Course_Name
C01	DBMS
C02	AI

A query can retrieve the student name and course name:

```
SELECT Student.Name, Course.Course_Name
FROM Student
JOIN Course
ON Student.Course_ID = Course.Course_ID;
```

Result

Name	Course_Name
Ram	DBMS
Sita	AI

Main points:

1. Uses more than one table.
2. Tables are generally related using keys.
3. `JOIN` is commonly used.

4. ON specifies the joining condition.
5. Table names can be used to identify columns clearly.

Conclusion

Multi-table queries allow information from related tables to be combined and retrieved as a single result.

C. LONG QUESTIONS — 15 MARKS

Q15. Explain SQL and Its Types of Commands with Suitable Examples. [15 Marks] ★ ★ ★

Introduction

SQL (Structured Query Language) is a standard language used to communicate with relational database systems. It is used to define database structures, manipulate data, retrieve information, and control access to database objects.

SQL commands can be categorized into **DDL, DML, and DCL**.

1. DDL — Data Definition Language

DDL is used to define and modify the structure of database objects.

CREATE

Used to create a database object such as a table.

```
CREATE TABLE Student (  
    Student_ID INT,  
    Name VARCHAR(50),  
    Age INT  
);
```

ALTER

Used to modify the structure of an existing table.

```
ALTER TABLE Student  
ADD Address VARCHAR(100);
```

DROP

Used to remove a database object.

```
DROP TABLE Student;
```

TRUNCATE

Used to remove all records from a table while retaining the table structure.

```
TRUNCATE TABLE Student;
```

2. DML — Data Manipulation Language

DML is used to retrieve and manipulate data stored in tables.

SELECT

Used to retrieve data.

```
SELECT * FROM Student;
```

INSERT

Used to add new records.

```
INSERT INTO Student  
VALUES (1, 'Ram', 20);
```

UPDATE

Used to modify existing records.

```
UPDATE Student  
SET Age = 21  
WHERE Student_ID = 1;
```

DELETE

Used to remove records.

```
DELETE FROM Student  
WHERE Student_ID = 1;
```

3. DCL — Data Control Language

DCL is used to control access and permissions.

GRANT

Provides privileges to a user.

```
GRANT SELECT ON Student TO user1;
```

REVOKE

Removes previously granted privileges.

```
REVOKE SELECT ON Student FROM user1;
```

Comparison

Type Purpose	Main Commands
DDL Defines database structure	CREATE, ALTER, DROP, TRUNCATE
DML Retrieves/manipulates data	SELECT, INSERT, UPDATE, DELETE
DCL Controls access	GRANT, REVOKE

Conclusion

SQL provides a complete mechanism for working with relational databases. **DDL manages structure, DML manages data, and DCL manages access and permissions.**

Q16. Explain SQL Querying Techniques with Suitable Examples. [15 Marks] ★ ★ ★

SQL provides several techniques for selecting and retrieving specific information from database tables.

Consider:

STUDENT			
ID	Name	Age	City
1	Ram	20	Itahari
2	Sita	22	Dharan
3	Hari	19	Itahari
4	Rita	25	Biratnagar

1. Selecting Rows

```
SELECT * FROM Student  
WHERE age > 20;
```

Returns students whose age is greater than 20.

2. Comparison Operators

```
SELECT * FROM Student  
WHERE age >= 20;
```

Common operators:

= < > <= >= <>

3. AND

Both conditions must be true.

```
SELECT * FROM Student  
WHERE age > 18 AND city = 'Itahari';
```

4. OR

At least one condition must be true.

```
SELECT * FROM Student  
WHERE city = 'Itahari' OR city = 'Dharan';
```

5. NOT

Negates a condition.

```
SELECT * FROM Student  
WHERE NOT city = 'Itahari';
```

6. BETWEEN

Selects values within a range.

```
SELECT * FROM Student
WHERE age BETWEEN 20 AND 25;
```

7. IN

Matches values from a specified list.

```
SELECT * FROM Student
WHERE city IN ('Itahari', 'Dharan');
```

8. LIKE

Performs pattern matching.

```
SELECT * FROM Student
WHERE name LIKE 'R%';
```

This finds names beginning with R.

9. IS NULL

Checks for missing/NULL values.

```
SELECT * FROM Student
WHERE phone IS NULL;
```

10. ORDER BY

Sorts query results.

```
SELECT * FROM Student
ORDER BY age DESC;
```

11. DISTINCT

Removes duplicate values from the result.

```
SELECT DISTINCT city
FROM Student;
```

Conclusion

SQL provides powerful search conditions and operators that allow users to retrieve **specific, sorted, filtered, and non-duplicate data** from databases.

Q17. Explain Subqueries in SQL with Examples. [15 Marks] ★ ★ ★

Introduction

A **subquery** is a query written inside another SQL query. The inner query is called the **subquery**, while the outer query uses the result produced by it.

Subqueries are useful when the result of one query is required to perform another query.

General Syntax

```
SELECT columns
FROM table
WHERE column operator
      (SELECT column
       FROM table
       WHERE condition);
```

Example 1: Subquery with Comparison

Suppose we want to find students whose age is greater than the average age.

```
SELECT name, age
FROM Student
WHERE age >
      (SELECT AVG(age)
       FROM Student);
```

The inner query:

```
SELECT AVG(age) FROM Student;
```

calculates the average age.

The outer query then finds students whose age is greater than that value.

Example 2: Subquery with IN

Suppose Student and Course tables are related.

```
SELECT name
FROM Student
WHERE Course_ID IN
      (SELECT Course_ID
       FROM Course
       WHERE Course_Name = 'DBMS');
```

The inner query finds the Course_ID for DBMS, and the outer query finds students enrolled in that course.

Example 3: Subquery with MAX

Find students whose age is equal to the maximum age:

```
SELECT name, age
FROM Student
WHERE age =
      (SELECT MAX(age)
       FROM Student);
```

Advantages of Subqueries

1. Can solve complex queries.
2. Makes it possible to use the result of one query in another.
3. Can reduce the need for multiple separate queries.
4. Useful with comparison and membership operators.
5. Helps retrieve information based on calculated or related values.

Conclusion

Subqueries provide a powerful method of performing complex database queries by allowing one query to use the result of another query.

D. SQL EXAM PRACTICE — MUST KNOW

These are the SQL patterns you should be able to write without looking at notes.

1. Display all records

```
SELECT * FROM Student;
```

2. Select specific columns

```
SELECT name, age
FROM Student;
```

3. Use WHERE

```
SELECT * FROM Student
WHERE age > 20;
```

4. Use AND

```
SELECT * FROM Student
WHERE age > 18 AND city = 'Itahari';
```

5. Use OR

```
SELECT * FROM Student
WHERE city = 'Itahari' OR city = 'Dharan';
```

6. Use BETWEEN

```
SELECT * FROM Student
WHERE age BETWEEN 18 AND 25;
```

7. Use IN

```
SELECT * FROM Student
WHERE city IN ('Itahari', 'Dharan');
```

8. Use LIKE

```
SELECT * FROM Student
WHERE name LIKE 'A%';
```

9. Check NULL

```
SELECT * FROM Student
WHERE phone IS NULL;
```

10. Sort records

```
SELECT * FROM Student
ORDER BY age DESC;
```

11. Remove duplicates

```
SELECT DISTINCT city
FROM Student;
```

12. Multi-table query

```
SELECT Student.Name, Course.Course_Name
FROM Student
JOIN Course
```

```
ON Student.Course_ID = Course.Course_ID;
```

13. Subquery

```
SELECT name  
FROM Student  
WHERE age >  
      (SELECT AVG(age)  
       FROM Student);
```

UNIT 5 — CONCEPTUAL DESIGN

A. VERY SHORT QUESTIONS — 2 MARKS

Q1. What is Conceptual Database Design? [2 Marks]

Conceptual database design is the process of developing a high-level representation of the data requirements of an organization. It identifies entities, attributes, relationships, and constraints without considering physical storage details.

Q2. What is Requirement Analysis? [2 Marks]

Requirement analysis is the process of identifying and documenting the data and information requirements of users and the organization before designing the database.

Q3. What is a Relation? [2 Marks]

A **relation** is a table in the relational model consisting of rows and columns, where each row represents a record and each column represents an attribute.

Q4. What is the Conceptual Design Process? [2 Marks]

The **conceptual design process** converts an organization's data requirements into a high-level database model by identifying entities, attributes, relationships, and constraints.

B. SHORT QUESTIONS — 7 MARKS

Q5. Explain the Conceptual Database Design Process. [7 Marks] ★ ★ ★

Conceptual database design creates a high-level model of the database based on organizational requirements. It focuses on **what data is required**, rather than how the data will be physically stored.

Main steps:

1. **Collect Requirements**
Gather information about the organization's data requirements from users and other sources.
2. **Identify Entities**
Identify important real-world objects about which data needs to be stored.

Example: Student, Course, Teacher.
3. **Identify Attributes**
Identify the properties that describe each entity.

Example: Student_ID, Name, Address.
4. **Identify Relationships**
Determine how different entities are related.

Example: Student **enrolls in** Course.
5. **Identify Constraints**
Determine rules and restrictions that the data must follow.
6. **Create Conceptual Model**
Represent the identified entities, attributes, and relationships using an ER model/diagram.
7. **Review and Validate the Model**
Check the model with users to ensure that it correctly represents their requirements.

Conclusion

Conceptual design provides a **clear blueprint of the required data and relationships** before the database is implemented.

Q6. Explain Requirement Analysis in Database Design.

[7 Marks] ★ ★ ★

Requirement analysis is the process of determining what data the organization needs and how that data will be used.

Main activities:

1. **Identify Users:**
Determine who will use the database.

2. **Collect Information:**
Gather information about current operations, documents, forms, and data requirements.
3. **Identify Data Requirements:**
Determine what information needs to be stored.
4. **Identify Operations:**
Determine how users will retrieve, insert, update, and use the data.
5. **Identify Relationships:**
Determine how different data items are related.
6. **Identify Constraints:**
Identify business rules and restrictions that data must satisfy.
7. **Validate Requirements:**
Discuss the requirements with users and confirm that they are complete and correct.

Conclusion

Requirement analysis ensures that the database design is based on **actual user and organizational needs**, reducing errors during later stages of development.

Q7. Explain the Identification of Relations in Database Design. [7 Marks]

Identification of relations involves determining how the identified data should be grouped into related structures for a relational database.

Main steps:

1. **Identify entities:**
Determine the major objects involved in the system.
2. **Identify attributes:**
Determine the properties of each entity.
3. **Group related attributes:**
Place attributes belonging to the same entity together.
4. **Identify primary keys:**
Select attributes that can uniquely identify records.
5. **Identify relationships:**
Determine connections between entities.
6. **Identify foreign keys:**
Determine attributes required to establish relationships between tables.
7. **Review the relations:**
Check that the relations correctly represent the organization's requirements.

Example

For a college database:

```
STUDENT
-----
Student_ID
Name
Address
Course_ID

COURSE
-----
Course_ID
Course_Name
```

Here:

- Student_ID identifies a student.
- Course_ID identifies a course.
- Course_ID can establish the relationship between Student and Course.

Conclusion

Proper identification of relations provides the foundation for converting the conceptual model into an effective relational database.

C. LONG QUESTIONS — 15 MARKS

Q8. Explain the Conceptual Database Design Process in Detail. [15 Marks] ★ ★ ★

Introduction

Conceptual database design is the process of developing a high-level representation of an organization's data requirements. It focuses on identifying **what data is required and how different data items are related**, without considering the physical implementation of the database.

The conceptual design acts as a blueprint for later database design.

Steps of Conceptual Database Design

1. Requirement Collection

The first step is to collect information about the organization's data requirements.

Information may be collected from:

- Users
 - Existing documents
 - Forms
 - Reports
 - Existing systems
 - Business processes
-

2. Requirement Analysis

The collected information is analyzed to determine:

- What data is required
 - Who will use the data
 - How the data will be used
 - What operations are required
 - What rules must be followed
-

3. Identify Entities

Important real-world objects or concepts are identified as entities.

For example, in a college system:

STUDENT
TEACHER
COURSE
DEPARTMENT

4. Identify Attributes

The properties describing each entity are identified.

For example:

STUDENT
↓
Student_ID
Name
Address
Age

5. Identify Relationships

The relationships between entities are determined.

For example:

STUDENT — ENROLLS — COURSE

This shows that a student enrolls in a course.

6. Identify Constraints

Rules that the database must follow are identified.

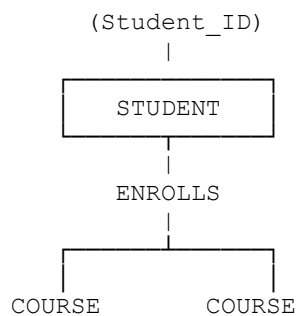
Examples:

- Student ID must be unique.
 - A course must have a valid Course ID.
 - Required information cannot be missing.
-

7. Develop the Conceptual Model

The identified entities, attributes, relationships, and constraints are represented using an **ER model/ER diagram**.

Example:



The conceptual model provides a graphical view of the required database structure.

8. Validate the Model

The conceptual model is reviewed with users and stakeholders to determine whether it correctly represents their requirements.

Errors, missing data, and incorrect relationships are corrected.

Conceptual Design Flow

```
Requirement Collection
      ↓
Requirement Analysis
      ↓
Identify Entities
      ↓
Identify Attributes
      ↓
Identify Relationships
      ↓
Identify Constraints
      ↓
Create Conceptual/ER Model
      ↓
Validate Model
```

Importance of Conceptual Design

1. Provides a clear view of data requirements.
2. Helps identify entities and relationships.
3. Improves communication between users and designers.
4. Detects missing or incorrect requirements early.
5. Provides a foundation for logical database design.
6. Reduces database design errors.
7. Separates data requirements from physical storage details.

Conclusion

Conceptual database design is an important early stage of database development. It transforms user requirements into a clear conceptual model containing **entities, attributes, relationships, and constraints**, which later serves as the foundation for logical and physical database design.

Q9. Explain Requirement Analysis and Identification of Relations in Database Design. [15 Marks] ★ ★ ★

Introduction

Before creating a database, it is necessary to understand the organization's requirements and determine how the required data should be organized. This involves **requirement analysis** followed by identification of relations.

Part A — Requirement Analysis

Requirement analysis identifies the data and processing needs of users.

1. Identify Users

Determine the people or groups who will use the database.

2. Collect Requirements

Information is collected through sources such as:

- Interviews
- Existing documents
- Forms
- Reports
- Existing systems
- Observation of business processes

3. Identify Required Data

Determine what information must be stored.

For a college:

- Student information
- Course information
- Teacher information
- Enrollment information

4. Identify Data Usage

Determine how users will use the stored information.

Examples:

- Searching records
- Adding new records
- Updating information

- Generating reports

5. Identify Relationships

Determine how different data items are related.

Example:

Student → enrolls in → Course

6. Identify Constraints

Identify rules that must be applied to the data.

Example:

Student_ID must uniquely identify each student.

7. Validate Requirements

Requirements should be reviewed with users to ensure that they are correct and complete.

Part B — Identification of Relations

After identifying requirements, the data is organized into suitable relations.

1. Identify Entities

Example:

STUDENT
COURSE

2. Identify Attributes

STUDENT
Student_ID
Name
Address
COURSE
Course_ID
Course_Name

3. Identify Primary Keys

Choose attributes that uniquely identify each record.

Student_ID → Primary Key
Course_ID → Primary Key

4. Identify Relationships

Determine relationships between the entities.

STUDENT — ENROLLS — COURSE

5. Identify Foreign Keys

A foreign key can be used to connect related tables.

Example:

```
STUDENT
Student_ID
Name
Course_ID ← Foreign Key
```

Example of Relations

```
STUDENT
+-----+-----+-----+
| Student_ID | Name   | Course_ID |
+-----+-----+-----+
| 101        | Ram    | C01       |
| 102        | Sita   | C02       |
+-----+-----+-----+
```

```
COURSE
+-----+-----+
| Course_ID | Course_Name |
+-----+-----+
| C01       | DBMS        |
| C02       | AI          |
+-----+-----+
```

Here, Student_ID uniquely identifies students, while Course_ID connects the Student and Course information.

Conclusion

Requirement analysis determines **what the database must contain and how it will be used**, while identification of relations organizes that information into suitable related structures. Together, they provide a strong foundation for database design.

UNIT 6 — LOGICAL DESIGN

Syllabus: Logical Design Process, Designing Entity Relationship Diagram, Creating Tables and Constraints, Creating Referential Keys.

Priority: ★ ★ ★ Important

A. VERY SHORT QUESTIONS — 2 MARKS

1. What is Logical Database Design?

Logical database design is the process of converting the conceptual database model into a logical structure such as **tables, attributes, primary keys and foreign keys**.

2. What is a Constraint?

A constraint is a rule imposed on database data to maintain **accuracy, validity and integrity**.

Examples: PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK.

3. What is a Primary Key?

A primary key is an attribute or set of attributes that **uniquely identifies each record** in a table.

Example:

```
Student_ID INT PRIMARY KEY
```

4. What is a Foreign Key?

A foreign key is an attribute in one table that refers to the **primary key of another table**.

5. What is Referential Integrity?

Referential integrity ensures that a foreign-key value must either **match an existing primary-key value** in the referenced table or be `NULL` when permitted.

6. What is an ER Diagram?

An ER diagram is a graphical representation of **entities, attributes and relationships** in a database.

7. What is a Relational Schema?

A relational schema describes the structure of a relation/table, including its **attributes, keys and constraints**.

B. SHORT QUESTIONS — 7 MARKS

★ 1. Explain the Logical Database Design Process.

Logical design converts the conceptual design/ER model into a relational database structure.

Steps:

1. Review requirements

Analyze the information and operations identified during requirement analysis.

2. Identify entities

Determine the major entities that need to be represented as tables.

3. Identify attributes

Determine the properties of each entity.

4. Select primary keys

Choose an attribute or combination of attributes that uniquely identifies each record.

5. Convert relationships

Convert relationships between entities into appropriate relational structures.

6. Create foreign keys

Add foreign keys to represent relationships between tables.

7. Define constraints

Apply constraints such as primary key, foreign key, unique and check constraints.

8. Validate the design

Check whether the resulting tables correctly represent the requirements and maintain data integrity.

Simple flow:

```
Requirements
  ↓
Identify Entities
  ↓
Identify Attributes
  ↓
Select Primary Keys
  ↓
Convert Relationships
```

↓
Create Foreign Keys
↓
Define Constraints
↓
Validate Database Design

★ 2. Explain Designing an Entity Relationship Diagram.

An ER diagram represents the logical structure of a database using entities, attributes and relationships.

Main steps:

1. **Identify entities** required by the system.
2. **Identify attributes** belonging to each entity.
3. Select the **primary key** of each entity.
4. Identify relationships between entities.
5. Determine the **cardinality** of relationships.
6. Add appropriate constraints.
7. Review and validate the ER diagram.

Example:

+-----+		+-----+
STUDENT		COURSE
+-----+		+-----+
Student_ID	-----	Course_ID
Name	ENROLLS	Course_Name
Course_ID		+-----+
+-----+		

Here:

- Student_ID → Primary Key of STUDENT
 - Course_ID → Primary Key of COURSE
 - Course_ID in STUDENT → Foreign Key
 - ENROLLS → Relationship
-

★ 3. Explain Creating Tables and Constraints.

After designing the logical structure, entities are converted into tables.

Steps:

1. Create table

Each entity is generally represented as a table.

2. Define attributes

Attributes become columns.

3. Select primary key

A primary key uniquely identifies every record.

4. Define data types

Each attribute is assigned a suitable data type.

5. Apply constraints

Constraints maintain data integrity.

Common constraints:

Constraint	Purpose
------------	---------

PRIMARY KEY	Uniquely identifies records
-------------	-----------------------------

FOREIGN KEY	Establishes relationship between tables
-------------	---

NOT NULL	Prevents NULL values
----------	----------------------

UNIQUE	Prevents duplicate values
--------	---------------------------

CHECK	Restricts values according to a condition
-------	---

Example:

```
CREATE TABLE Student (  
    Student_ID INT PRIMARY KEY,  
    Name VARCHAR(50) NOT NULL,  
    Age INT CHECK (Age >= 18),  
    Email VARCHAR(100) UNIQUE  
);
```

★ 4. Explain Referential Keys / Foreign Keys.

A referential key, commonly called a **foreign key**, is used to establish a relationship between two tables.

The foreign key in one table refers to the primary key of another table.

Example:

```
CREATE TABLE Course (  
    Course_ID INT PRIMARY KEY,  
    Course_Name VARCHAR(50)  
);  
  
CREATE TABLE Student (  
    Student_ID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Course_ID INT,  
    FOREIGN KEY (Course_ID)  
        REFERENCES Course(Course_ID)  
);
```

Here:

```
COURSE  
Course_ID ← Primary Key  
    ↑  
    |  
    | Referenced by  
    |  
STUDENT  
Course_ID ← Foreign Key
```

Importance:

1. Establishes relationships between tables.
2. Maintains referential integrity.
3. Prevents invalid references.
4. Reduces data inconsistency.
5. Represents relationships from the ER model.
6. Helps maintain the logical structure of the database.

C. LONG QUESTIONS — 15 MARKS

★ ★ ★ 1. Explain the Logical Database Design Process in Detail.

Introduction

Logical database design is the process of transforming the conceptual database model into a **logical relational structure** consisting of tables, attributes, keys, relationships and constraints.

The main objective is to create a database structure that correctly represents the requirements while maintaining data integrity.

Logical Design Process

1. Analyze Requirements

First, identify the data required by the system and the operations that will be performed on that data.

2. Identify Entities

Identify the important objects or entities that need to be stored.

For example:

```
Student
Course
Teacher
```

Each major entity generally becomes a table.

3. Identify Attributes

Identify the properties of each entity.

For example:

```
STUDENT
-----
Student_ID
Name
Address
Email
```

4. Select Primary Keys

Select an attribute that uniquely identifies each record.

Example:

```
Student_ID → Primary Key
```

5. Identify Relationships

Determine how entities are related to one another.

Example:

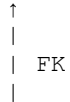
```
STUDENT ---- ENROLLS ---- COURSE
```

6. Convert Relationships into Tables/Foreign Keys

Relationships are represented using foreign keys or separate relationship tables depending on the relationship structure.

Example:

```
COURSE
Course_ID (PK)
Course_Name
```



```
STUDENT
Student_ID (PK)
Name
Course_ID (FK)
```

7. Define Constraints

Apply appropriate constraints to maintain data integrity.

Important constraints include:

- Primary Key
- Foreign Key
- NOT NULL
- UNIQUE
- CHECK

8. Validate the Logical Design

The completed design should be checked against the original requirements to ensure that all required information and relationships are represented correctly.

Overall Process

```
Requirement Analysis
↓
Identify Entities
↓
Identify Attributes
↓
Select Primary Keys
↓
Identify Relationships
↓
Create Tables
↓
Create Foreign Keys
↓
Apply Constraints
```

↓
Validate Design

Example

Suppose a college stores student and course information.

```
COURSE
-----
Course_ID (PK)
Course_Name

          1
          |
          | offers/enrolls
          |
          N

STUDENT
-----
Student_ID (PK)
Name
Course_ID (FK)
```

Corresponding tables:

```
CREATE TABLE Course (
    Course_ID INT PRIMARY KEY,
    Course_Name VARCHAR(50)
);

CREATE TABLE Student (
    Student_ID INT PRIMARY KEY,
    Name VARCHAR(50),
    Course_ID INT,
    FOREIGN KEY (Course_ID)
        REFERENCES Course(Course_ID)
);
```

Conclusion

Logical database design converts the conceptual model into a structured relational database by defining **tables, attributes, keys, relationships and constraints**. A properly designed logical model improves consistency and maintains data integrity.

★ ★ ★ 2. Explain Tables, Constraints and Referential Keys with an Example.

Introduction

In a relational database, data is organized into tables. Keys and constraints are used to identify records, establish relationships and maintain the correctness of data.

1. Creating Tables

An entity is generally represented as a table, while its attributes become columns.

Example:

```
STUDENT
-----
Student_ID
Name
Email
Course_ID
```

SQL:

```
CREATE TABLE Student (
    Student_ID INT PRIMARY KEY,
    Name VARCHAR(50),
    Email VARCHAR(100),
    Course_ID INT
);
```

2. Creating Constraints

Constraints are rules that restrict the data stored in a table.

Primary Key

Uniquely identifies each record.

```
Student_ID INT PRIMARY KEY
```

NOT NULL

Does not allow a column to contain NULL values.

```
Name VARCHAR(50) NOT NULL
```

UNIQUE

Prevents duplicate values.

```
Email VARCHAR(100) UNIQUE
```

CHECK

Ensures that values satisfy a specified condition.

```
Age INT CHECK (Age >= 18)
```

Foreign Key

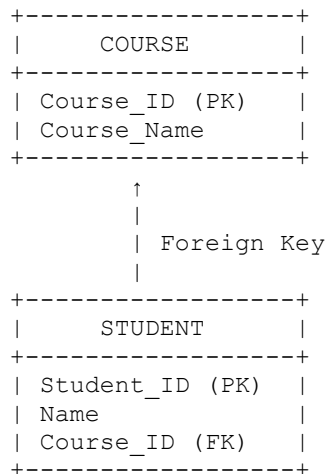
Connects one table to another.

```
FOREIGN KEY (Course_ID)
REFERENCES Course (Course_ID)
```

3. Creating Referential Keys

A foreign key is used to establish a relationship between two tables.

Example:



SQL:

```
CREATE TABLE Course (
    Course_ID INT PRIMARY KEY,
    Course_Name VARCHAR(50)
);

CREATE TABLE Student (
    Student_ID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Course_ID INT,
    FOREIGN KEY (Course_ID)
        REFERENCES Course (Course_ID)
);
```

Benefits of Referential Keys

1. Establish relationships between tables.
2. Maintain referential integrity.
3. Prevent invalid references.
4. Reduce data inconsistency.
5. Represent relationships between entities.
6. Improve database organization.

Conclusion

Tables store the data, constraints maintain its validity, and foreign keys establish relationships between tables. Together they form an important part of logical database design.

D. ★ ★ QUICK 2-MARK REVISION

Memorize these before the exam:

- **Logical Design:** Converts conceptual model into tables, keys and constraints.
- **Table:** Collection of related data organized in rows and columns.
- **Primary Key:** Uniquely identifies each record.
- **Foreign Key:** References a primary key in another table.
- **Constraint:** Rule used to maintain data integrity.
- **Referential Integrity:** Ensures valid relationships between related tables.
- **ER Diagram:** Graphical representation of entities, attributes and relationships.

UNIT 7 — NORMALIZATION

Syllabus: Use of Normalization, Different Forms of Normalization

Priority: ★ ★ ★ Important

Exam Focus: 1NF, 2NF, 3NF + need/advantages of normalization

A. VERY SHORT QUESTIONS — 2 MARKS

1. What is Normalization?

Normalization is the process of organizing data in a database into well-structured tables to **reduce data redundancy and avoid data anomalies**.

2. Why is Normalization Used?

Normalization is used to:

- Reduce data redundancy.
 - Remove data inconsistency.
 - Avoid insertion, deletion and update anomalies.
 - Improve database structure.
-

3. What is Data Redundancy?

Data redundancy means **unnecessary repetition of the same data** in multiple places in a database.

4. What is Functional Dependency?

Functional dependency is a relationship where the value of one attribute determines the value of another attribute.

It is represented as:

$A \rightarrow B$

Meaning: **A determines B**.

5. What is 1NF?

A relation is in **First Normal Form (1NF)** when every attribute contains **atomic/single values** and there are no repeating groups.

6. What is 2NF?

A relation is in **Second Normal Form (2NF)** when it is in 1NF and every non-key attribute is **fully dependent on the whole primary key**.

7. What is 3NF?

A relation is in **Third Normal Form (3NF)** when it is in 2NF and has **no transitive dependency** of non-key attributes on the primary key.

8. What is an Anomaly?

An anomaly is an undesirable problem that occurs because of poor database design.

Main types:

- Insertion anomaly
 - Update anomaly
 - Deletion anomaly
-

B. SHORT QUESTIONS — 7 MARKS

★ 1. Explain the Need and Advantages of Normalization.

Normalization organizes a database into properly structured relations.

Need for Normalization

1. Reduces data redundancy

It minimizes unnecessary duplication of data.

2. Prevents update anomalies

A change in data does not need to be made repeatedly in many places.

3. Prevents insertion anomalies

New information can be inserted without requiring unrelated information.

4. Prevents deletion anomalies

Deleting one record does not accidentally remove other important information.

5. Improves data consistency

The same information is less likely to have different values in different locations.

6. Improves database organization

Data is divided into logical and meaningful tables.

7. Maintains data integrity

Proper relationships and dependencies make the database more reliable.

Main benefits:

```
Normalization
↓
Less Redundancy
↓
Fewer Anomalies
↓
Better Consistency
↓
Better Data Integrity
```

★ 2. Explain First Normal Form (1NF).

A table is in **First Normal Form (1NF)** when:

1. Each column contains **atomic values**.
2. There are no repeating groups.
3. Each field contains a single value.

Not in 1NF:

Student_ID Name Courses

101 Ram DBMS, OS

The `Courses` field contains multiple values.

Converted to 1NF:

Student_ID Name Course

101 Ram DBMS

101 Ram OS

Each field now contains a single value.

Key point:

1NF = Atomic values + No repeating groups

★ 3. Explain Second Normal Form (2NF).

A relation is in **2NF** if:

1. It is already in **1NF**.
2. Every non-key attribute is fully dependent on the **entire primary key**.
3. There should be no **partial dependency**.

Example:

Consider:

`ENROLLMENT(Student_ID, Course_ID, Student_Name, Course_Name, Grade)`

Suppose the primary key is:

`(Student_ID, Course_ID)`

Dependencies:

`Student_ID → Student_Name`
`Course_ID → Course_Name`

These attributes depend only on part of the composite key. Therefore, partial dependency exists.

Convert into 2NF:

`STUDENT`

Student_ID (PK)
Student_Name

COURSE
Course_ID (PK)
Course_Name

ENROLLMENT
Student_ID (FK)
Course_ID (FK)
Grade

Now non-key attributes depend on the complete key of their respective relations.

Key point:

2NF = 1NF + No Partial Dependency

★ 4. Explain Third Normal Form (3NF).

A relation is in **3NF** if:

1. It is already in **2NF**.
2. There is no **transitive dependency**.
3. Non-key attributes should depend directly on the primary key.

Example:

STUDENT(Student_ID, Student_Name, Department_ID, Department_Name)

Dependencies:

Student_ID → Department_ID
Department_ID → Department_Name

Therefore:

Student_ID → Department_Name

indirectly.

This is a **transitive dependency**.

Convert into 3NF:

STUDENT
Student_ID (PK)
Student_Name
Department_ID (FK)

DEPARTMENT
Department_ID (PK)
Department_Name

Key point:

3NF = 2NF + No Transitive Dependency

C. LONG QUESTIONS — 15 MARKS

★ ★ ★ 1. What is Normalization? Explain 1NF, 2NF and 3NF with Examples.

Introduction

Normalization is a database design technique used to organize data into properly structured relations. Its main purpose is to **reduce redundancy, eliminate anomalies and improve data integrity**.

The commonly discussed normal forms are **1NF, 2NF and 3NF**.

1. First Normal Form (1NF)

A relation is in 1NF when every attribute contains **only atomic values** and there are no repeating groups.

Example — Before 1NF:

Student_ID Name Courses

101 Ram DBMS, OS

The `Courses` column contains multiple values.

After 1NF:

Student_ID Name Course

101 Ram DBMS

Student_ID Name Course

101 Ram OS

Now each field contains a single value.

Rule:

1NF $\rightarrow$ Atomic values + No repeating groups

2. Second Normal Form (2NF)

A relation is in 2NF when:

- It is already in 1NF.
- Every non-key attribute is fully dependent on the entire primary key.
- There is no partial dependency.

Consider:

ENROLLMENT
(Student_ID, Course_ID, Student_Name, Course_Name, Grade)

Primary key:

(Student_ID, Course_ID)

Dependencies:

Student_ID $\rightarrow$ Student_Name
Course_ID $\rightarrow$ Course_Name
(Student_ID, Course_ID) $\rightarrow$ Grade

Student_Name **depends only on** Student_ID, and Course_Name **depends only on** Course_ID.

Therefore, partial dependency exists.

Decomposition:

STUDENT

Student_ID (PK)
Student_Name

COURSE

Course_ID (PK)
Course_Name

```
ENROLLMENT
-----
Student_ID (FK)
Course_ID (FK)
Grade
```

The partial dependencies are removed.

Rule:

2NF $\rightarrow$ 1NF + No Partial Dependency

3. Third Normal Form (3NF)

A relation is in 3NF when:

- It is already in 2NF.
- It contains no transitive dependency.
- Non-key attributes depend directly on the primary key.

Consider:

```
STUDENT
(Student_ID, Student_Name, Department_ID, Department_Name)
```

Dependencies:

```
Student_ID  $\rightarrow$  Department_ID
Department_ID  $\rightarrow$  Department_Name
```

Thus:

```
Student_ID  $\rightarrow$  Department_Name
```

through Department_ID.

This is transitive dependency.

Decomposition:

```
STUDENT
-----
Student_ID (PK)
Student_Name
Department_ID (FK)

DEPARTMENT
-----
Department_ID (PK)
Department_Name
```

Now the transitive dependency is removed.

Rule:

3NF $\rightarrow$ 2NF + No Transitive Dependency

Comparison of Normal Forms

Normal Form Main Requirement

1NF Atomic values; no repeating groups

2NF 1NF + no partial dependency

3NF 2NF + no transitive dependency

Normalization Flow

UNNORMALIZED
↓
1NF
(Atomic Values)
↓
2NF
(No Partial Dependency)
↓
3NF
(No Transitive Dependency)

Advantages of Normalization

1. Reduces data redundancy.
2. Eliminates insertion anomalies.
3. Eliminates update anomalies.
4. Eliminates deletion anomalies.
5. Improves data consistency.
6. Improves data integrity.
7. Produces a well-organized database.

Conclusion

Normalization systematically organizes database relations by removing unnecessary dependencies and redundancy. **1NF removes repeating/multiple values, 2NF removes partial dependency, and 3NF removes transitive dependency.**

★ ★ ★ 2. Explain Database Anomalies and How Normalization Removes Them.

Poorly designed tables can produce several problems known as **data anomalies**.

1. Insertion Anomaly

An insertion anomaly occurs when it is impossible or difficult to insert new information without adding unrelated data.

Example:

If student and course information are stored together, a new course may not be added until at least one student enrolls in it.

Normalization separates the entities into appropriate tables, allowing the course to be inserted independently.

2. Update Anomaly

An update anomaly occurs when the same information is stored in multiple records and must be changed in several places.

Example:

If a department name is repeated for 100 students, changing the department name requires updating many records.

Normalization stores department information separately.

```
STUDENT
Student_ID
Department_ID
```

```
DEPARTMENT
Department_ID
Department_Name
```

Now the department name needs to be changed only once.

3. Deletion Anomaly

A deletion anomaly occurs when deleting one piece of information unintentionally removes another useful piece of information.

Example:

If the only student enrolled in a course is deleted, the course information may also disappear from the table.

Normalization separates student and course information so that deleting a student does not delete the course.

How Normalization Helps

```
Poor Database Design
    ↓
Redundancy
    ↓
Insertion / Update / Deletion Anomalies
    ↓
Normalization
    ↓
Separate Related Data
    ↓
Better Consistency & Integrity
```

Conclusion

Normalization minimizes redundancy and separates logically different data, thereby reducing insertion, update and deletion anomalies.

UNIT 8 — DATABASE TECHNOLOGY

Syllabus: Different Database Technologies, Database Client & Server Processing, OLAP & OLTP Database Techniques

Priority: ★ ★ Important

A. VERY SHORT QUESTIONS — 2 MARKS

1. What is Database Technology?

Database technology refers to the methods, systems and tools used to **store, manage, process and retrieve data efficiently**.

2. What is Client-Server Database Processing?

Client-server database processing is an architecture where the **client requests data or services** and the database server processes the request and provides the result.

3. What is a Database Client?

A database client is an application or computer that **sends requests to a database server** to access or manipulate data.

4. What is a Database Server?

A database server is a computer or software system that **stores and manages databases** and responds to requests from clients.

5. What is OLTP?

OLTP (Online Transaction Processing) is a database technique designed to handle a large number of **day-to-day operational transactions** quickly and accurately.

Examples: ATM transactions, online banking, sales transactions.

6. What is OLAP?

OLAP (Online Analytical Processing) is a database technique used to **analyze large amounts of data** for reporting, decision-making and business analysis.

7. Give two differences between OLTP and OLAP.

OLTP

Handles daily transactions

Frequently updated

OLAP

Performs data analysis

Mainly used for analysis/reporting

B. SHORT QUESTIONS — 7 MARKS

★ 1. Explain Client-Server Database Processing.

Client-server database processing divides database operations between **clients and servers**.

Components

1. Client

The client is the front-end application through which users interact with the database.

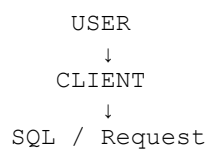
Examples:

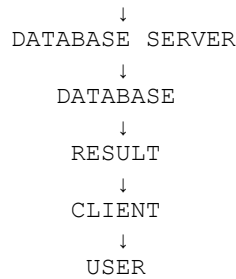
- Web application
- Desktop application
- Mobile application

2. Database Server

The server stores the database and processes queries received from clients.

Working





Process

1. User performs an operation through the client.
2. Client sends a request/query to the database server.
3. Server receives and processes the request.
4. Server accesses the required database.
5. Result is returned to the client.
6. Client displays the result to the user.

Advantages

- Centralized database management.
 - Better security.
 - Easier maintenance.
 - Multiple clients can access the same database.
 - Reduces data duplication.
 - Efficient resource sharing.
-

★ 2. Explain OLTP.

OLTP (Online Transaction Processing) is a database technique used to process routine and real-time business transactions.

Characteristics

1. Handles a large number of transactions.
2. Transactions are usually short.
3. Requires fast response time.
4. Data is frequently inserted, updated or deleted.
5. Maintains high data accuracy and consistency.
6. Supports multiple concurrent users.
7. Used for day-to-day operations.

Examples

- ATM transactions
- Online banking

- Airline ticket booking
- Point-of-sale systems
- Online shopping orders
- Hotel reservations

Example

When a customer purchases a product online:

```

Customer
  ↓
Place Order
  ↓
OLTP System
  ↓
Update Order
  ↓
Update Inventory
  ↓
Record Payment
  
```

All these operations are processed as transactions.

★ 3. Explain OLAP.

OLAP (Online Analytical Processing) is a technology used to analyze large volumes of data from different perspectives.

It is mainly used for **decision-making, reporting and business analysis**.

Characteristics

1. Handles large volumes of historical data.
2. Used mainly for analysis.
3. Supports complex queries.
4. Helps identify trends and patterns.
5. Data is usually read more than updated.
6. Supports management decision-making.
7. Allows analysis from multiple dimensions.

Example

A company may use OLAP to analyze:

```

Sales
├── By Year
├── By Month
├── By Product
└── By Region
  
```

└ By Customer

Management can use this information to determine which products or regions generate the highest sales.

★ 4. Differentiate Between OLTP and OLAP.

Basis	OLTP	OLAP
Full form	Online Transaction Processing	Online Analytical Processing
Purpose	Daily transactions	Data analysis
Data	Current/operational	Historical/analytical
Operations	Insert, Update, Delete	Mainly Read/Analyze
Queries	Simple and short	Complex
Users	Clerks, customers, operational staff	Managers, analysts
Response	Very fast	May involve longer processing
Example	ATM, banking, sales	Business intelligence, reporting

Easy memory:

OLTP → TRANSACTION
OLAP → ANALYSIS

C. LONG QUESTIONS — 15 MARKS

★ ★ ★ 1. Explain OLTP and OLAP Database Techniques with Differences and Examples.

Introduction

OLTP and OLAP are two important database processing techniques designed for different purposes.

- **OLTP** is mainly used for processing daily transactions.
 - **OLAP** is mainly used for analyzing data and supporting decision-making.
-

OLTP — Online Transaction Processing

OLTP is designed to process routine and real-time transactions efficiently.

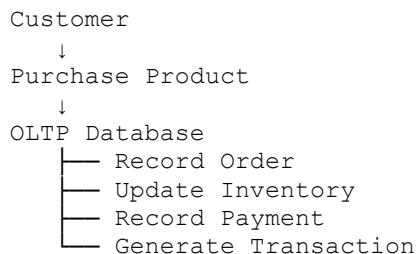
Characteristics of OLTP

1. Handles large numbers of transactions.
2. Transactions are generally short.
3. Provides fast response.
4. Data is frequently inserted, updated and deleted.
5. Maintains data consistency.
6. Supports multiple concurrent users.
7. Primarily works with current operational data.

Examples

- Banking transactions
- ATM systems
- Online shopping
- Ticket booking
- Point-of-sale systems
- Reservation systems

OLTP Example



OLAP — Online Analytical Processing

OLAP is designed to analyze large amounts of data and provide useful information for decision-making.

Characteristics of OLAP

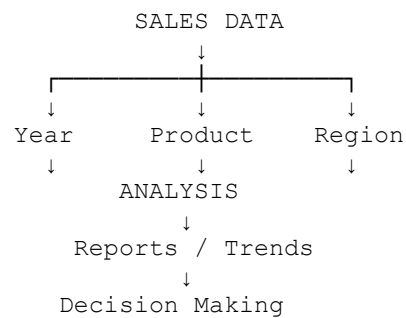
1. Works with large amounts of data.

2. Uses historical data.
3. Supports complex queries.
4. Used for analysis and reporting.
5. Helps identify trends and patterns.
6. Supports management decision-making.
7. Data is generally read and analyzed rather than frequently modified.

Examples

- Sales analysis
- Financial analysis
- Business intelligence
- Market analysis
- Management reporting

OLAP Example



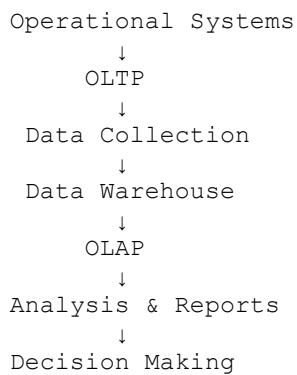
OLTP vs OLAP

Feature	OLTP	OLAP
Purpose	Transaction processing	Data analysis
Full form	Online Transaction Processing	Online Analytical Processing
Main users	Operational users	Managers/analysts
Data	Current operational data	Historical/analytical data
Queries	Simple	Complex
Transactions	Very frequent	Less frequent
Data modification	Frequent	Less frequent

Feature	OLTP	OLAP
Main operation	Insert/Update/Delete	Read/Analyze
Response	Very fast	Relatively longer
Objective	Run daily operations	Support decisions
Example	ATM/banking	Sales analysis

Relationship

In many organizations:



Conclusion

OLTP and OLAP serve different purposes. **OLTP focuses on fast and reliable transaction processing**, while **OLAP focuses on analyzing large amounts of data for reporting and decision-making**.

★ ★ ★ 2. Explain Client-Server Database Architecture and its Working.

Introduction

Client-server database architecture is a database processing model in which database operations are divided between a **client** and a **database server**.

Components

1. Client

The client is the application through which users interact with the system.

It:

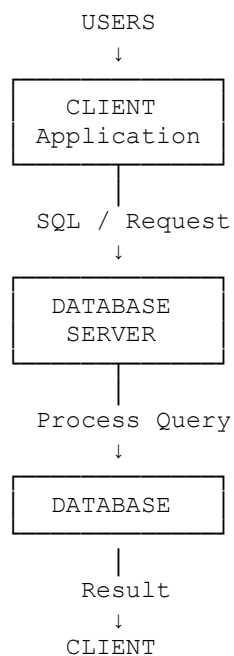
- Accepts user input.
- Sends requests to the server.
- Receives results.
- Displays results to the user.

2. Database Server

The database server:

- Stores the database.
- Receives client requests.
- Processes SQL queries.
- Manages data access.
- Returns results.

Architecture



Working

1. User sends a request through the client application.
2. Client prepares and sends the request to the database server.
3. Server receives the request.
4. Database server processes the SQL query.

5. Server accesses the required data.
6. The result is sent back to the client.
7. Client presents the result to the user.

Advantages

- Centralized data management.
- Improved security.
- Easy database maintenance.
- Supports multiple users.
- Reduces duplication.
- Efficient sharing of database resources.

Conclusion

Client-server architecture separates user interaction from database management. The client handles the user interface while the server manages and processes database operations.

UNIT 9 — DISTRIBUTED ARCHITECTURE

Syllabus: Distributed Database Architecture, Advantages and Disadvantages of Distributed Database

Priority: ★ ★ Important

A. VERY SHORT QUESTIONS — 2 MARKS

1. What is a Distributed Database?

A distributed database is a collection of logically related data that is **physically stored at multiple locations connected through a network**.

2. What is a Distributed Database Management System (DDBMS)?

A DDBMS is a database management system that manages data distributed across **multiple network-connected sites** while making it appear as a single database to users.

3. What is a Distributed Database Architecture?

Distributed database architecture is the structure used to organize and manage **databases located at different network sites**.

4. What is a Database Site?

A site is a computer or location in a distributed system that can **store or process part of the database**.

5. What is Data Distribution?

Data distribution is the process of storing database data across **multiple sites or locations** in a distributed system.

6. What is Data Fragmentation?

Data fragmentation is the process of dividing a database relation into **smaller parts called fragments** that can be stored at different sites.

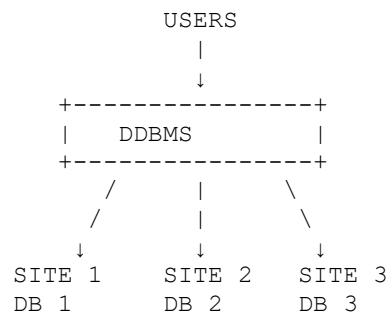
B. SHORT QUESTIONS — 7 MARKS

★ 1. Explain Distributed Database Architecture.

A distributed database architecture consists of multiple database sites connected through a communication network.

Although the data is physically distributed, users can access it as if it were part of **one logical database**.

Basic Architecture



Each site can store and process part of the overall database.

Main Components

1. Users/Applications

Users access the distributed database through applications.

2. Communication Network

Connects the different database sites.

3. Database Sites

Individual locations that store or process database data.

4. Local DBMS

Manages the database stored at a particular site.

5. Distributed DBMS

Coordinates operations among different sites.

Working

1. User sends a database request.
 2. DDBMS determines where the required data is located.
 3. Requests are sent to the appropriate sites.
 4. Local databases process their parts of the request.
 5. Results are coordinated by the DDBMS.
 6. Final result is returned to the user.
-

★ 2. Explain the Advantages of Distributed Database.

Advantages:

1. Improved reliability

Failure at one site may not completely stop the entire system.

2. Better availability

Data can remain available through other sites when one site fails.

3. Faster local access

Users can access data stored near their location more efficiently.

4. Scalability

New sites can be added as the organization grows.

5. Resource sharing

Different locations can share data and database resources.

6. Reduced communication cost

Local operations can be performed without always communicating with a central site.

7. Supports geographically distributed organizations

Branches located in different areas can maintain and access shared data.

★ 3. Explain the Disadvantages of Distributed Database.

Disadvantages:

1. Complexity

Designing and managing a distributed database is more complex than a centralized database.

2. Higher cost

Additional hardware, network infrastructure and software may be required.

3. Security issues

Data distributed across multiple sites creates additional security challenges.

4. Network dependency

Communication between sites depends on a reliable network.

5. Data consistency problems

Keeping copies of data synchronized can be difficult.

6. Difficult maintenance

Managing multiple database sites requires more administration.

7. Failure management is complex

Detecting and recovering from failures across multiple sites can be difficult.

C. LONG QUESTIONS — 15 MARKS

★ ★ ★ 1. What is a Distributed Database? Explain its Architecture, Advantages and Disadvantages.

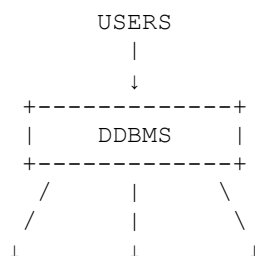
Introduction

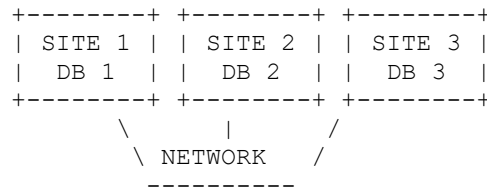
A distributed database is a collection of logically related data that is **physically distributed across multiple locations connected by a computer network**.

A Distributed Database Management System (DDBMS) coordinates these databases and provides users with access to the distributed data.

Distributed Database Architecture

In a distributed database system, several database sites communicate through a network.





Components

1. Database Sites

Each site contains a database or a portion of the distributed database.

2. Local DBMS

Each site may have a local DBMS responsible for managing its local data.

3. Distributed DBMS

The DDBMS coordinates operations involving multiple sites.

4. Communication Network

The network allows different database sites to communicate and exchange data.

5. Users and Applications

Users access the distributed database through applications without necessarily knowing the physical location of the data.

Working of Distributed Database

```

User Request
  ↓
DDBMS receives request
  ↓
Identify required site(s)
  ↓
Send request through network
  ↓
Local DBMS processes request
  ↓
Results are coordinated
  ↓
Final result returned to user
  
```

Advantages

1. Improved Reliability

If one site becomes unavailable, other sites may continue operating.

2. High Availability

Data and services can remain available at multiple locations.

3. Faster Local Access

Users can access locally stored data without always depending on a remote central server.

4. Scalability

Additional sites can be added as the organization expands.

5. Resource Sharing

Different branches or locations can share information.

6. Geographical Distribution

It is suitable for organizations operating across multiple cities, regions or countries.

7. Reduced Central Bottleneck

Processing can be distributed among multiple sites instead of relying completely on one central database.

Disadvantages

1. Complexity

Designing and managing multiple interconnected databases is complicated.

2. Higher Cost

More servers, networking equipment and administration may be required.

3. Security Problems

Protecting data across multiple locations is more challenging.

4. Network Dependency

Network failure can affect communication between sites.

5. Data Consistency

Maintaining consistent copies of data across different sites can be difficult.

6. Difficult Maintenance

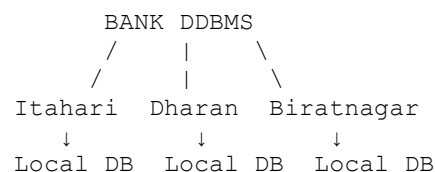
Database administration becomes more complicated because multiple sites must be monitored.

7. Complex Recovery

Failure detection and recovery may require coordination between several sites.

Example

Consider a bank with branches in:



Each branch can maintain local information while the DDBMS allows authorized users to access information across branches.

Conclusion

Distributed database architecture stores logically related data across multiple network-connected sites. It provides **availability, scalability, local access and resource sharing**, but introduces challenges such as **complexity, security, network dependency and consistency management**.

D. ★★ IMPORTANT COMPARISON

Centralized vs Distributed Database

Basis	Centralized Database	Distributed Database
Storage	Single location	Multiple locations
Network dependency	Lower	Higher
Management	Simpler	More complex
Local access	May require remote access	Often faster locally
Failure	Central failure can affect whole system	One site may fail independently
Scalability	Comparatively limited	Easier to expand across locations
Cost	Generally lower	Generally higher

Complex + Costly + Security + Network Dependency + Consistency Problems

UNIT 10 — DATABASE EVALUATION AND TRANSACTION

Syllabus: Transaction Management, ACID Properties, Database Evaluation Process

Total Hours: 12

Priority: ★ ★ ★ Important

A. VERY SHORT QUESTIONS — 2 MARKS

1. What is a Transaction?

A transaction is a **logical unit of database operations** that performs one complete task.

Example: Transferring money from one bank account to another.

2. What is Transaction Management?

Transaction management is the process of controlling database transactions to ensure **correctness, consistency and reliability** of the database.

3. What are ACID Properties?

ACID properties are four properties that ensure reliable database transactions:

- **A — Atomicity**
 - **C — Consistency**
 - **I — Isolation**
 - **D — Durability**
-

4. What is Atomicity?

Atomicity means a transaction is treated as a **single unit**. Either all its operations are completed or none of them are performed.

5. What is Consistency?

Consistency ensures that a transaction changes the database from **one valid state to another valid state**.

6. What is Isolation?

Isolation ensures that concurrently executing transactions do not improperly interfere with one another.

7. What is Durability?

Durability means that once a transaction is successfully committed, its changes are **permanently stored** even if a system failure occurs.

8. What is Commit?

`COMMIT` permanently saves the changes made by a transaction in the database.

9. What is Rollback?

`ROLLBACK` cancels the changes made by a transaction and returns the database to its previous state.

B. SHORT QUESTIONS — 7 MARKS

★ 1. Explain Transaction Management.

Transaction management controls the execution of database transactions and ensures that the database remains reliable and consistent.

Main functions:

1. Transaction execution

Manages the operations performed by a transaction.

2. Concurrency control

Controls simultaneous transactions so that they do not cause incorrect results.

3. Maintaining consistency

Ensures that database rules and constraints are maintained.

4. Commit management

A successful transaction is committed and its changes become permanent.

5. Rollback and recovery

If a transaction fails, its changes can be rolled back.

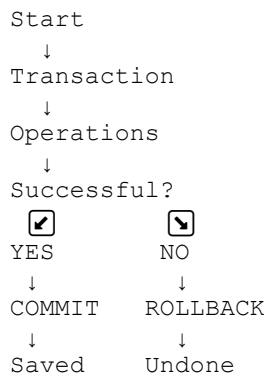
6. Failure handling

Helps recover the database from system or transaction failures.

7. Maintaining reliability

Ensures transactions follow the required properties of reliable processing.

Basic transaction flow:



★ 2. Explain ACID Properties of a Transaction.

ACID properties ensure that database transactions are processed reliably.

A — Atomicity

A transaction is treated as one indivisible unit.

All operations → Successful
OR
No operations → Performed

Example: In a bank transfer, money must be deducted from one account **and** added to another. If one operation fails, the entire transaction is rolled back.

C — Consistency

The transaction must maintain all database rules and constraints.

Valid State
↓
Transaction
↓
Valid State

I — Isolation

Each transaction should operate independently without being improperly affected by other concurrent transactions.

D — Durability

Once a transaction is committed, its changes remain permanently stored even after a system failure.

Memory:

A → All or Nothing

C → Correct State

I → Independent

D → Doesn't Disappear

★ 3. Explain the Database Evaluation Process.

Database evaluation is the process of assessing whether a database system is **performing effectively and meeting its requirements**.

Main steps:

1. Define evaluation objectives

Determine what aspects of the database need to be evaluated.

2. Evaluate performance

Check response time, processing speed and system performance.

3. Evaluate data quality

Check accuracy, consistency and completeness of data.

4. Evaluate security

Check whether unauthorized access and data-related risks are controlled.

5. Evaluate reliability

Determine whether the database operates correctly and consistently.

6. Identify problems

Find performance, security, data quality or operational problems.

7. Improve the system

Take corrective actions based on the evaluation results.

Basic process:

```
Set Objectives
  ↓
Evaluate Performance
  ↓
Check Data Quality
  ↓
Check Security & Reliability
  ↓
Identify Problems
  ↓
Take Corrective Action
```

C. LONG QUESTIONS — 15 MARKS

★ ★ ★ 1. What is a Transaction? Explain Transaction Management and ACID Properties.

Introduction

A transaction is a **logical unit of work** consisting of one or more database operations. A transaction must be completed correctly so that the database remains consistent and reliable.

For example, transferring money from Account A to Account B involves:

1. Deduct money from A
2. Add money to B

Both operations together form one transaction.

Transaction Management

Transaction management controls the execution of transactions and maintains database reliability.

1. Transaction Execution

The DBMS manages the operations performed during a transaction.

2. Concurrency Control

When multiple transactions execute at the same time, the DBMS controls their execution to avoid incorrect results.

3. Commit

If all operations are successful, the transaction is committed.

```
COMMIT;
```

The changes become permanent.

4. Rollback

If an error occurs, the transaction can be rolled back.

```
ROLLBACK;
```

The changes made by the unsuccessful transaction are undone.

5. Failure Recovery

Transaction management helps restore the database to a correct state after failures.

ACID Properties

1. Atomicity

Atomicity means **all operations of a transaction occur or none occur**.

Example:

```
Transfer Rs. 5,000
```

```
Account A: -Rs. 5,000
```

```
Account B: +Rs. 5,000
```

If adding money to B fails, the deduction from A must also be cancelled.

Atomicity = All or Nothing

2. Consistency

Consistency ensures that a transaction takes the database from one **valid state to another valid state**.

All integrity constraints must remain satisfied.

Consistency = Valid State → Valid State

3. Isolation

Isolation ensures that one transaction does not improperly interfere with another transaction executing at the same time.

For example, two users updating the same bank account should not produce an incorrect balance.

Isolation = Transactions work independently

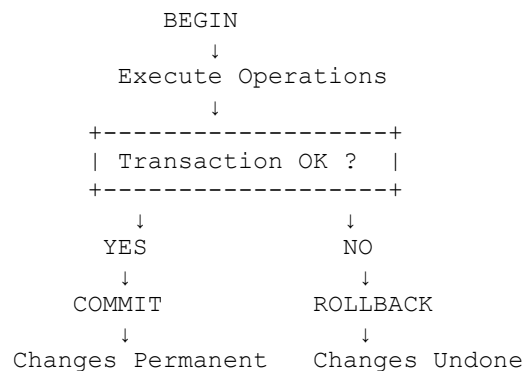
4. Durability

Durability guarantees that once a transaction is committed, its changes remain permanent.

Even if the system crashes immediately after the commit, the committed changes should not be lost.

Durability = Permanent Changes

Transaction Flow



ACID Summary

Property	Meaning
----------	---------

Atomicity	All operations or none
-----------	------------------------

Consistency	Maintains valid database state
-------------	--------------------------------

Isolation	Transactions do not improperly interfere
-----------	--

Durability	Committed changes are permanent
------------	---------------------------------

Conclusion

Transaction management ensures that database operations are executed safely and reliably. The **ACID properties—Atomicity, Consistency, Isolation and Durability—are essential for maintaining database correctness and reliability.**

★ ★ ★ 2. Explain the Database Evaluation Process in Detail.

Introduction

Database evaluation is the systematic process of checking whether a database system is **efficient, reliable, secure and capable of meeting its intended requirements.**

Database Evaluation Process

1. Define Evaluation Objectives

First, determine what aspects of the database system need to be evaluated.

Examples:

- Performance
 - Security
 - Reliability
 - Data quality
-

2. Evaluate Database Performance

Measure how efficiently the database processes requests.

Important factors include:

- Response time
 - Processing speed
 - Query performance
 - Resource utilization
-

3. Evaluate Data Quality

Check whether stored data is:

- Accurate
 - Complete
 - Consistent
 - Reliable
-

4. Evaluate Security

Check whether database information is protected against unauthorized access and misuse.

5. Evaluate Reliability

Determine whether the database can operate correctly and recover from failures.

6. Identify Problems

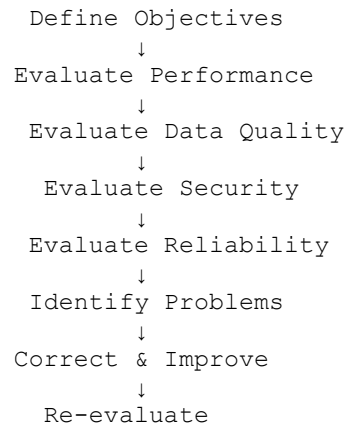
The evaluation should identify weaknesses such as:

- Slow queries
 - Data inconsistencies
 - Security weaknesses
 - Reliability issues
-

7. Take Corrective Actions

After identifying problems, improvements are made to the database system.

Process Diagram



Conclusion

Database evaluation helps determine whether a database system meets its performance, quality, security and reliability requirements. Continuous evaluation and improvement help maintain an effective database system.

D. ★★ IMPORTANT DIFFERENCES

COMMIT vs ROLLBACK

COMMIT

Saves transaction changes

Makes changes permanent

Used after successful transaction

ROLLBACK

Undoes transaction changes

Returns to previous state

Used when transaction fails or is cancelled

UNIT 11 — DATA ANALYSIS

Syllabus: Data Analysis Process, Types of Data Analysis, Data Analysis Steps

Total Hours: 12

Priority: ★ ★ Important

A. VERY SHORT QUESTIONS — 2 MARKS

1. What is Data Analysis?

Data analysis is the process of **examining, processing and interpreting data** to discover useful information, patterns and relationships.

2. What is the Data Analysis Process?

The data analysis process is a systematic procedure of **collecting, preparing, analyzing and interpreting data** to obtain meaningful information.

3. What is Descriptive Analysis?

Descriptive analysis summarizes **what has happened or what is present in the data** using tables, charts, averages and other summaries.

4. What is Diagnostic Analysis?

Diagnostic analysis examines data to determine **why something happened**.

5. What is Predictive Analysis?

Predictive analysis uses existing and historical data to **predict possible future outcomes**.

6. What is Prescriptive Analysis?

Prescriptive analysis recommends **possible actions or decisions** based on analyzed data.

B. SHORT QUESTIONS — 7 MARKS

★ 1. Explain the Data Analysis Process.

Data analysis follows a series of steps for converting raw data into useful information.

Main process:

1. Define the objective

Determine what problem needs to be solved or what information is required.

2. Collect data

Gather relevant data from databases, files, surveys or other sources.

3. Prepare the data

Organize the collected data and remove errors or unnecessary information.

4. Clean the data

Identify and handle missing, incorrect or duplicate data.

5. Analyze the data

Apply suitable techniques to identify patterns, relationships and trends.

6. Interpret the results

Determine what the analyzed results mean.

7. Present the results

Communicate findings using tables, charts, reports or other suitable methods.

Process:

```
Define Objective
  ↓
Collect Data
  ↓
Prepare Data
  ↓
Clean Data
  ↓
Analyze Data
  ↓
Interpret Results
  ↓
Present Results
```

★ 2. Explain the Types of Data Analysis.

The major types of data analysis are:

1. Descriptive Analysis

Describes what has happened or summarizes existing data.

Example:

Calculating the average sales of a company.

2. Diagnostic Analysis

Examines data to determine **why something happened**.

Example:

Finding why sales decreased in a particular month.

3. Predictive Analysis

Uses historical data to predict future events or outcomes.

Example:

Predicting next month's sales.

4. Prescriptive Analysis

Suggests possible actions based on the analysis.

Example:

Recommending which product should receive more marketing.

Easy flow:

Descriptive → What happened?
Diagnostic → Why did it happen?
Predictive → What may happen?
Prescriptive → What should we do?

★ 3. Explain the Steps of Data Analysis.

Step 1: Define the Problem

Clearly identify the problem or question that needs to be answered.

Step 2: Collect Data

Gather relevant data from appropriate sources.

Step 3: Organize Data

Arrange the collected data into a useful structure.

Step 4: Clean Data

Remove errors, duplicates and deal with missing or incorrect values.

Step 5: Analyze Data

Apply appropriate analytical methods to identify patterns and relationships.

Step 6: Interpret Results

Understand the meaning and significance of the results.

Step 7: Communicate Results

Present the findings through reports, tables, charts or visualizations.

Step 8: Make Decisions

Use the findings to support appropriate decisions or actions.

C. LONG QUESTIONS — 15 MARKS

★ ★ ★ 1. Explain the Data Analysis Process and its Steps in Detail.

Introduction

Data analysis is the process of examining and interpreting data to obtain **meaningful information that can support decision-making**.

A systematic data analysis process ensures that raw data is transformed into useful and reliable information.

1. Define the Objective

First, clearly identify the purpose of the analysis.

The analyst should determine:

- What problem needs to be solved?
 - What information is required?
 - What decision needs to be supported?
-

2. Collect Data

Relevant data is collected from appropriate sources.

Possible sources include:

- Databases
- Files
- Surveys
- Business systems
- Other available data sources

The collected data should be relevant to the objective.

3. Prepare and Organize Data

Raw data is organized into a suitable structure so that it can be analyzed effectively.

For example:

```
Raw Data
↓
Organized Tables
↓
Structured Dataset
```

4. Clean the Data

The data is checked for problems such as:

- Missing values
- Duplicate records
- Incorrect values
- Inconsistent formats

Cleaning improves the quality and reliability of the analysis.

5. Analyze the Data

Appropriate analytical techniques are applied to the prepared data.

The analyst may look for:

- Patterns
 - Trends
 - Relationships
 - Differences
 - Important findings
-

6. Interpret Results

The results are examined to understand what they mean in relation to the original objective.

The analyst should distinguish meaningful findings from irrelevant information.

7. Present the Results

Findings can be presented using:

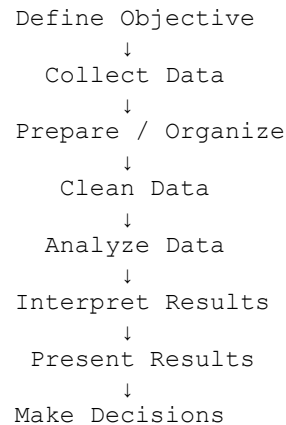
- Tables
- Charts
- Graphs
- Reports
- Summaries

The presentation should make the findings easy to understand.

8. Make Decisions

Finally, the results are used to support decisions or actions.

Complete Process:



Conclusion

The data analysis process converts raw data into useful information through systematic collection, preparation, cleaning, analysis and interpretation. It helps organizations make **better and more informed decisions**.

★ ★ ★ 2. Explain Different Types of Data Analysis with Examples.

Data analysis can be classified according to the type of question it answers.

1. Descriptive Analysis

Descriptive analysis summarizes existing data and explains **what happened or what is happening**.

Examples:

- Average monthly sales
- Number of customers
- Total annual revenue

Question answered:

“What happened?”

2. Diagnostic Analysis

Diagnostic analysis examines data to understand **why something happened**.

Examples:

- Why did sales decrease?
- Why did website traffic increase?
- Why did a product receive fewer orders?

Question answered:

“Why did it happen?”

3. Predictive Analysis

Predictive analysis uses historical data and patterns to estimate **what may happen in the future**.

Examples:

- Predicting future sales
- Predicting customer demand
- Predicting possible customer behavior

Question answered:

“What is likely to happen?”

4. Prescriptive Analysis

Prescriptive analysis uses analysis results to suggest **what action should be taken**.

Examples:

- Which product should be promoted?
- How much inventory should be maintained?
- Which marketing strategy should be selected?

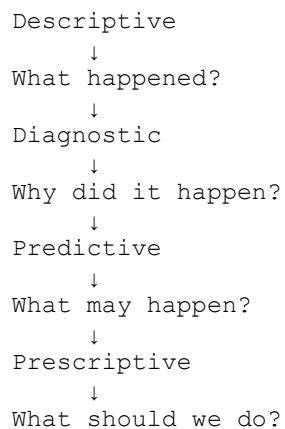
Question answered:

“What should we do?”

Comparison

Type	Main Question	Example
Descriptive	What happened?	Calculate previous sales
Diagnostic	Why did it happen?	Find reason for sales decline
Predictive	What may happen?	Forecast future sales
Prescriptive	What should we do?	Recommend a marketing strategy

Relationship:



Conclusion

The four major types of data analysis provide different levels of understanding. Descriptive analysis explains the past, diagnostic finds causes, predictive estimates future outcomes, and prescriptive recommends actions.

UNIT 12 — DATABASE AND THE WORLD WIDE WEB

Syllabus: Web Data Management, Web Search, Web Crawling

Total Hours: 8

Priority: ★ ★ Important

A. VERY SHORT QUESTIONS — 2 MARKS

1. What is Web Data Management?

Web data management is the process of **collecting, storing, organizing, retrieving and managing data available on the World Wide Web.**

2. What is Web Search?

Web search is the process of finding **relevant information or web pages on the Internet** using a search engine.

3. What is a Search Engine?

A search engine is a software system that helps users **find information on the Web** based on their search queries.

4. What is Web Crawling?

Web crawling is the automated process of **discovering and visiting web pages** to collect information for a search engine.

5. What is a Web Crawler?

A web crawler is an automated program that visits web pages, follows links and **collects information from websites.**

6. What is an Index in Web Search?

A search index is an organized collection of information about web pages that allows a search engine to **quickly retrieve relevant results**.

B. SHORT QUESTIONS — 7 MARKS

★ 1. Explain Web Data Management.

Web data management deals with managing the large amount of data available through the Web.

Main activities:

1. Data Collection

Collecting data from different websites and online sources.

2. Data Storage

Storing collected web data in databases or other storage systems.

3. Data Organization

Organizing data so that it can be efficiently accessed and processed.

4. Data Integration

Combining information obtained from different web sources.

5. Data Retrieval

Finding and retrieving required information from stored data.

6. Data Quality

Handling problems such as duplicate, incomplete or inconsistent information.

7. Data Security

Protecting web-related data from unauthorized access and misuse.

Basic process:

```
Web Data
↓
Collect
↓
Store
↓
Organize
↓
Process / Integrate
↓
Retrieve
```

★ 2. Explain Web Search and its Working.

Web search is the process of finding relevant information on the Internet using a search engine.

Working of Web Search

```
User enters query
      ↓
Search Engine
      ↓
Search Index
      ↓
Find Relevant Pages
      ↓
Rank Results
      ↓
Display Results
      ↓
User
```

Steps:

1. User enters a query

The user enters keywords or a question.

2. Search engine processes the query

The system analyzes the words in the query.

3. Search index is consulted

The search engine searches its index rather than manually searching every webpage.

4. Relevant pages are identified

Pages matching the query are selected.

5. Results are ranked

Relevant results are ranked according to the search engine's ranking process.

6. Results are displayed

The search engine displays the results to the user.

Example:

If the user searches:

```
"best database management system"
```

the search engine retrieves and displays relevant indexed web pages.

★ 3. Explain Web Crawling.

Web crawling is the automated process of discovering and visiting web pages on the Internet.

A program called a **web crawler** or **spider** performs the crawling.

Working:

```
Starting Web Page
↓
Read Page
↓
Collect Information
↓
Find Links
↓
Visit New Links
↓
Repeat
↓
Store Information in Index
```

Steps:

1. Crawler starts with known web pages or URLs.
2. It downloads the webpage.
3. It examines the content.
4. It identifies links on the page.
5. It follows relevant links to discover more pages.
6. Information about pages is collected.
7. The collected information is used to build or update a search index.

Importance

- Discovers new web pages.
 - Collects web information.
 - Helps search engines maintain their indexes.
 - Detects updated webpages.
 - Makes web search possible on a large scale.
-

C. LONG QUESTIONS — 15 MARKS

★ ★ ★ 1. Explain Web Data Management, Web Search and Web Crawling.

Introduction

The World Wide Web contains a huge amount of continuously changing information. Web data management, web search and web crawling are important techniques used to **collect, organize, discover and retrieve web information.**

1. Web Data Management

Web data management is concerned with managing data available through the Web.

Main activities:

- Collecting web data
- Storing data
- Organizing data
- Integrating information
- Retrieving information
- Maintaining data quality
- Protecting data

Process:

```
Collect
  ↓
Store
  ↓
Organize
  ↓
Integrate
  ↓
Process
  ↓
Retrieve
```

2. Web Search

Web search allows users to find relevant information from the Internet.

Working:



Main steps:

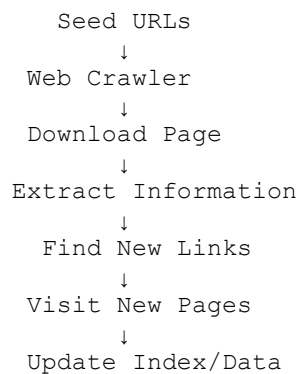
1. User enters a search query.
 2. Search engine processes the query.
 3. The query is matched against its index.
 4. Relevant pages are identified.
 5. Results are ranked.
 6. Results are presented to the user.
-

3. Web Crawling

Web crawling is the automated process of visiting webpages and discovering information and links.

A **web crawler** performs this task.

Working:

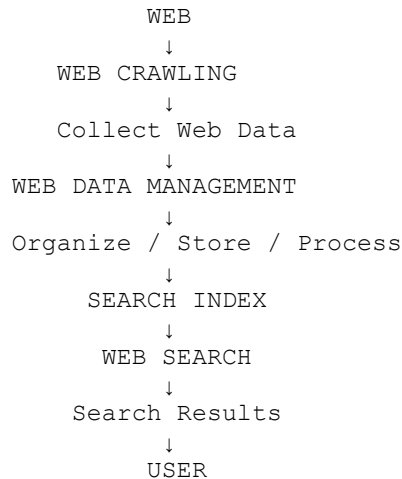


Importance of Web Crawling

1. Discovers webpages.
2. Collects information from webpages.
3. Helps search engines build indexes.

4. Finds newly created pages.
 5. Detects changes in existing pages.
 6. Provides data needed for web search.
-

Relationship Between the Three



Conclusion

Web crawling discovers and collects information from webpages, web data management organizes and processes that information, and web search allows users to retrieve relevant information. Together, they enable efficient access to the enormous amount of information available on the Web.

D. ★★ IMPORTANT DIFFERENCE

Web Crawling vs Web Search

Web Crawling

Discovers and visits webpages

Performed by crawlers/spiders

Collects webpage information

Helps build/update indexes

Web Search

Finds information for users

Performed through search engines

Retrieves relevant indexed information

Uses indexes to provide results

Web Crawling

Works largely automatically

Web Search

Usually begins with a user query

Crawler collects → Database/Index organizes → Search Engine retrieves